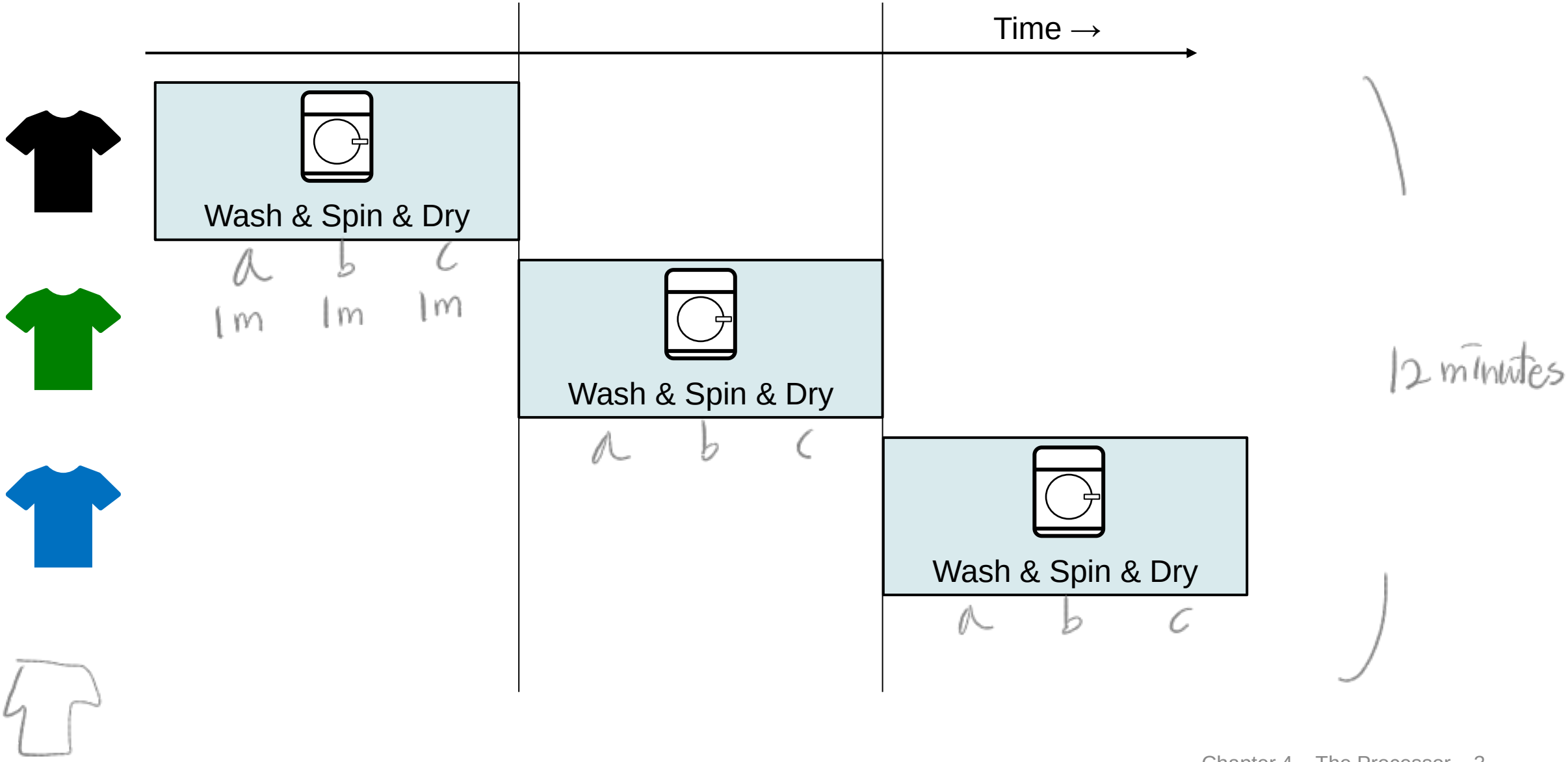
Introduction to Computer Architecture
Chapter 4 - 3

Pipelining

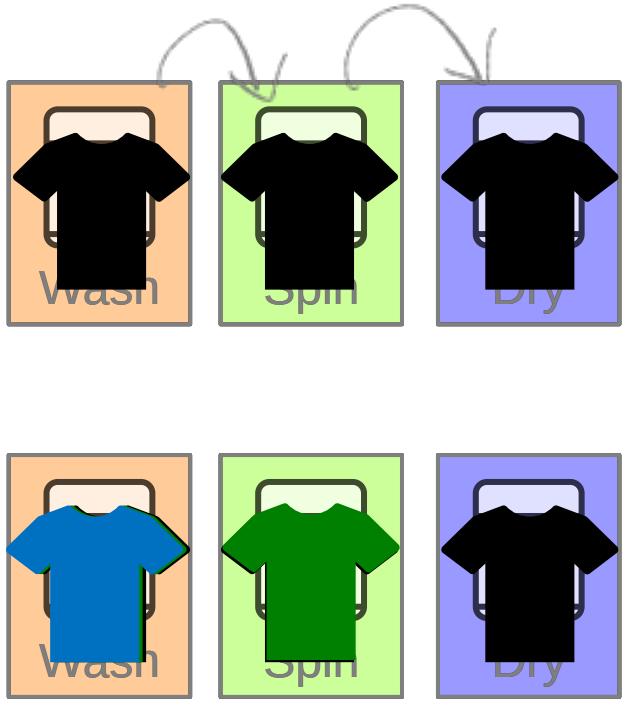
Hyungmin Cho

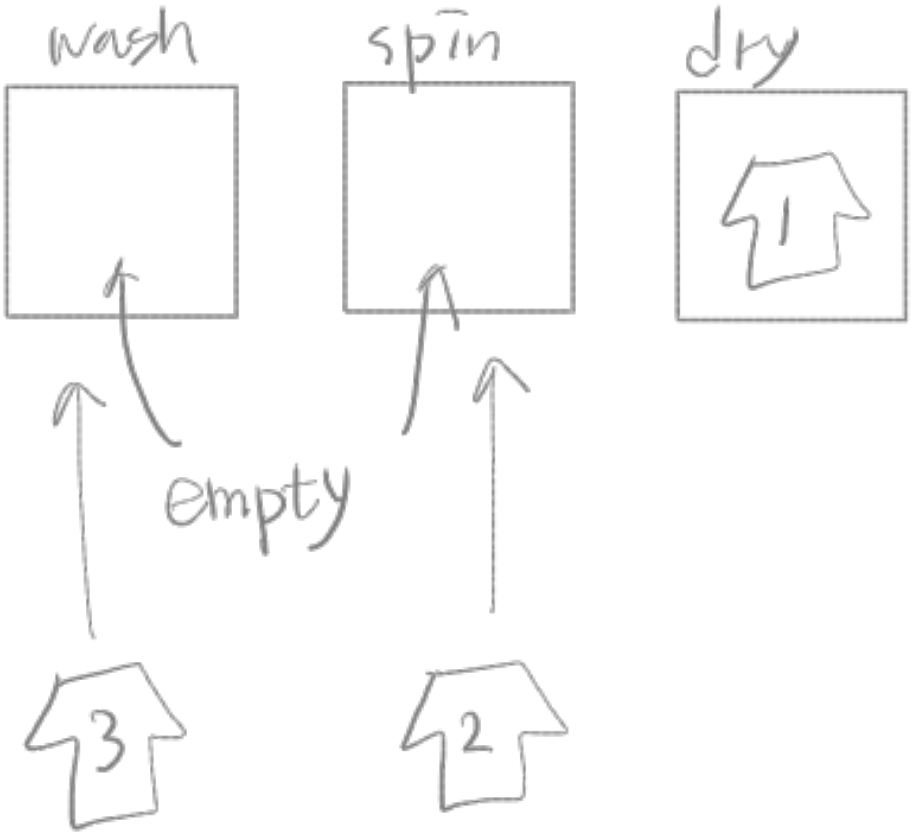
Department of Computer Science and Engineering
Sungkyunkwan University

Pipelining Analogy

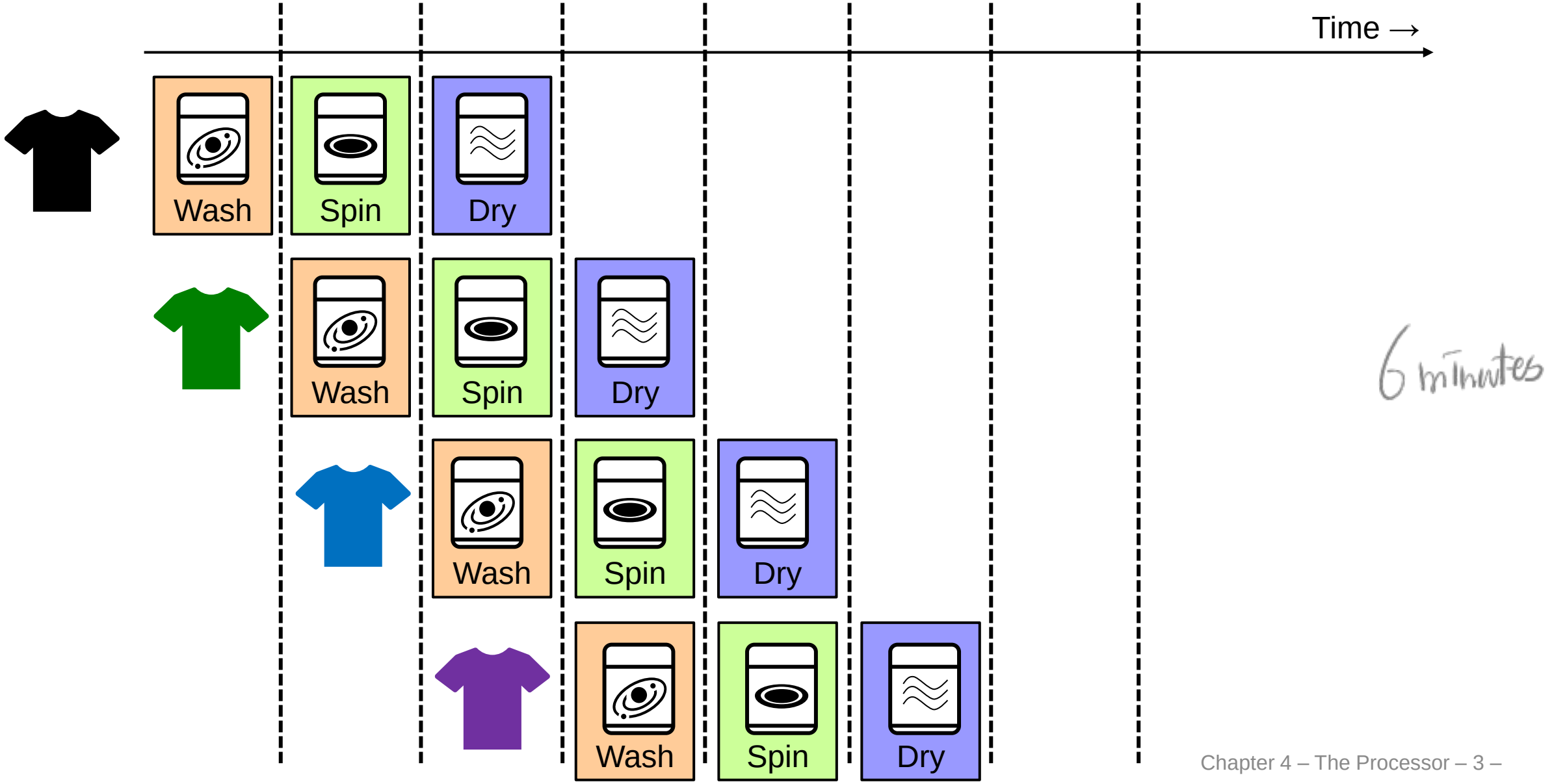


Pipelining Analogy

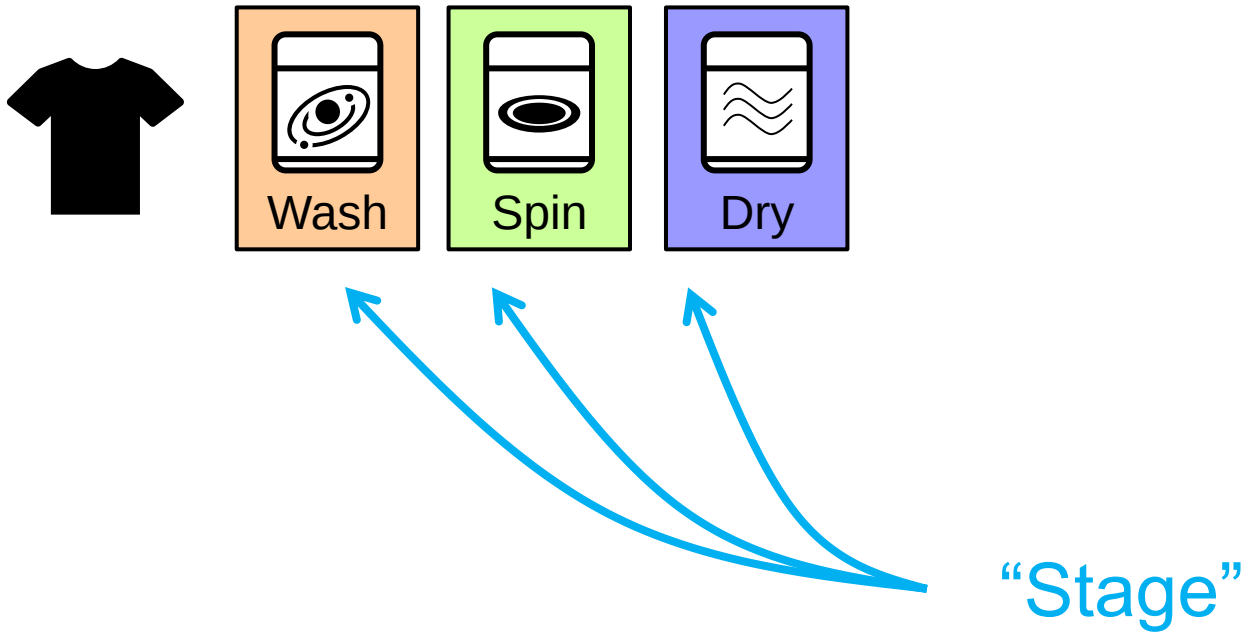




Pipelining Analogy



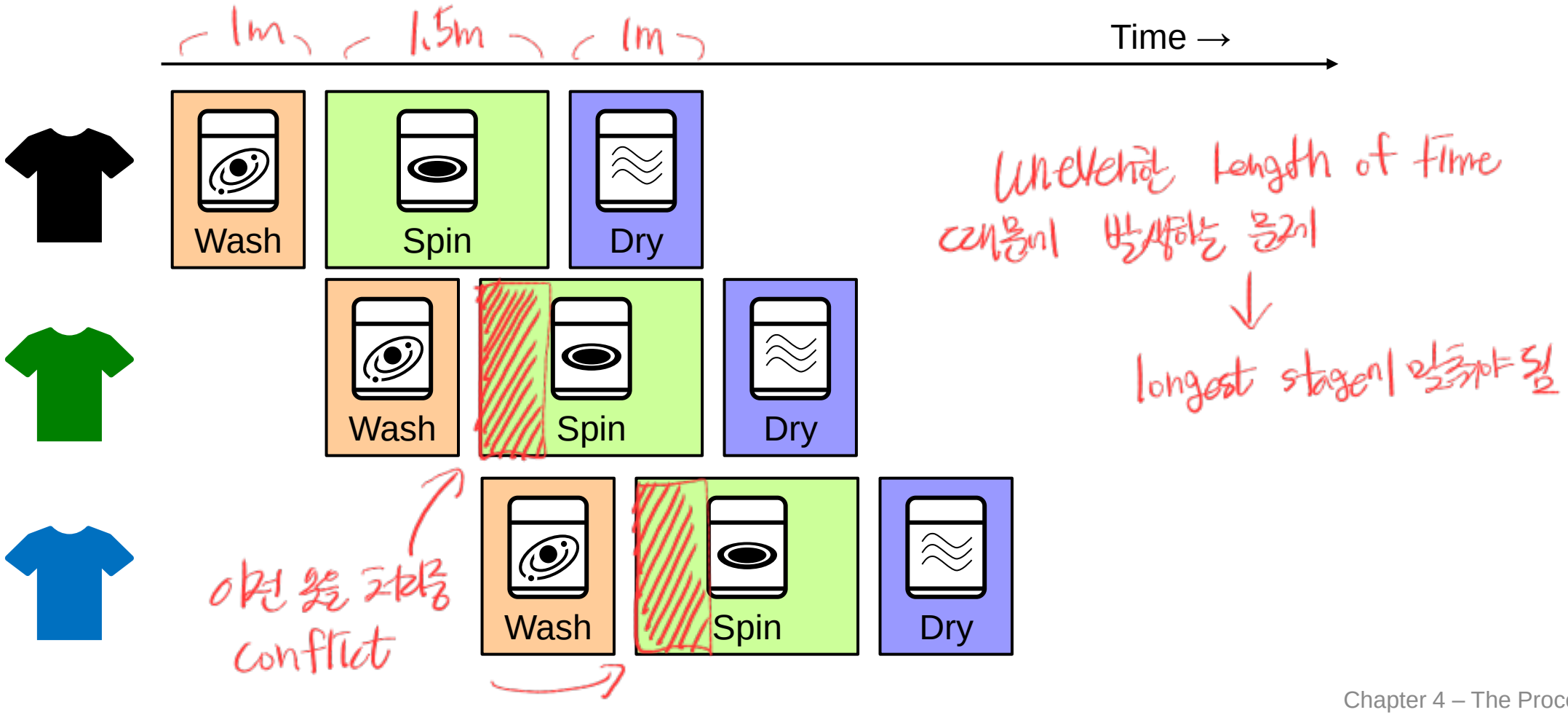
Pipelining Analogy



pipelining이란
더많은 resource 필요
(additional overhead)
그러나 overall processing speed는
향상 pipelining 필요

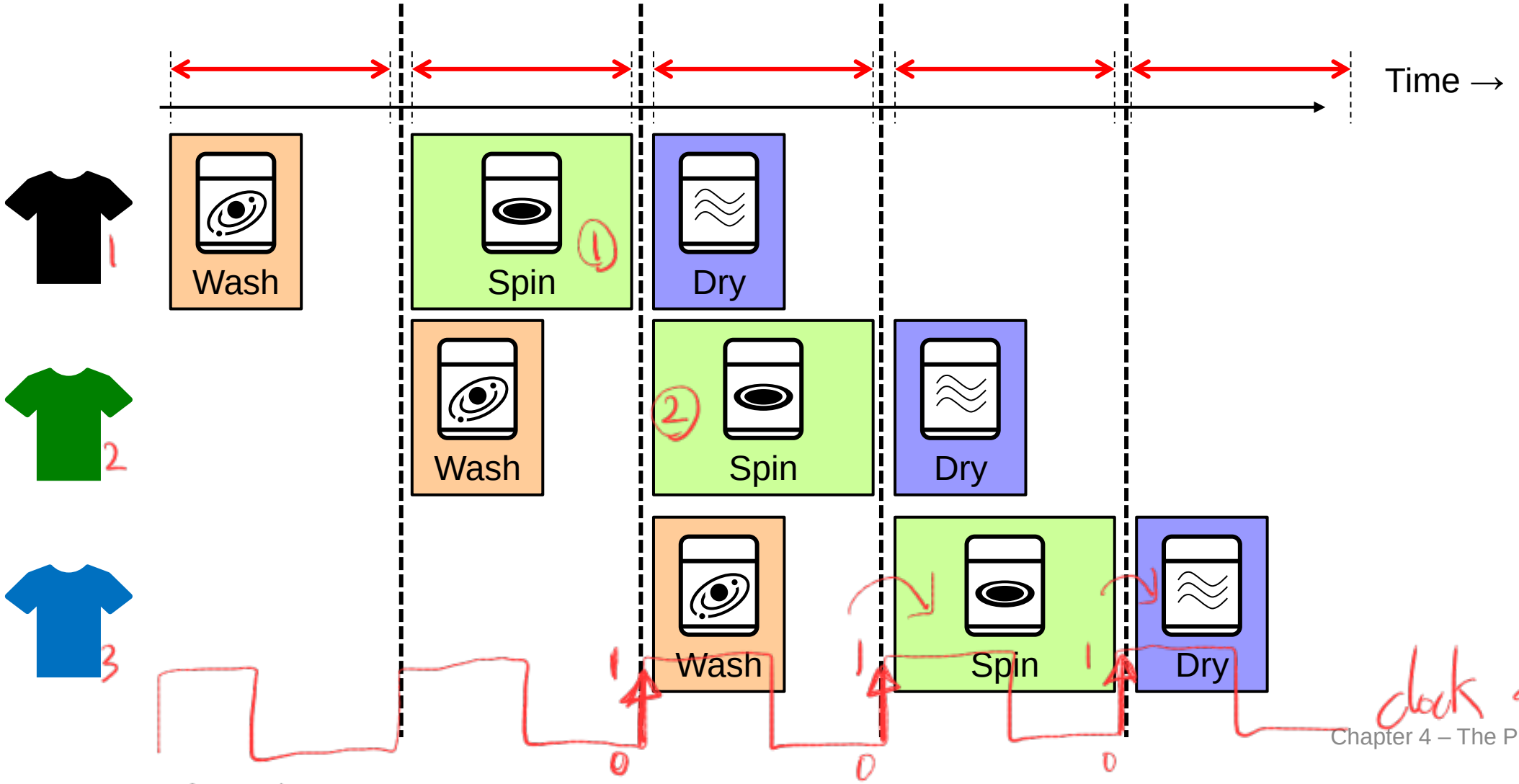
Clock Period in Pipeline

- All stages use the same clock input
- Determined by the longest stage



Clock Period in Pipeline

- Determined by the longest stage



Pipeline Performance

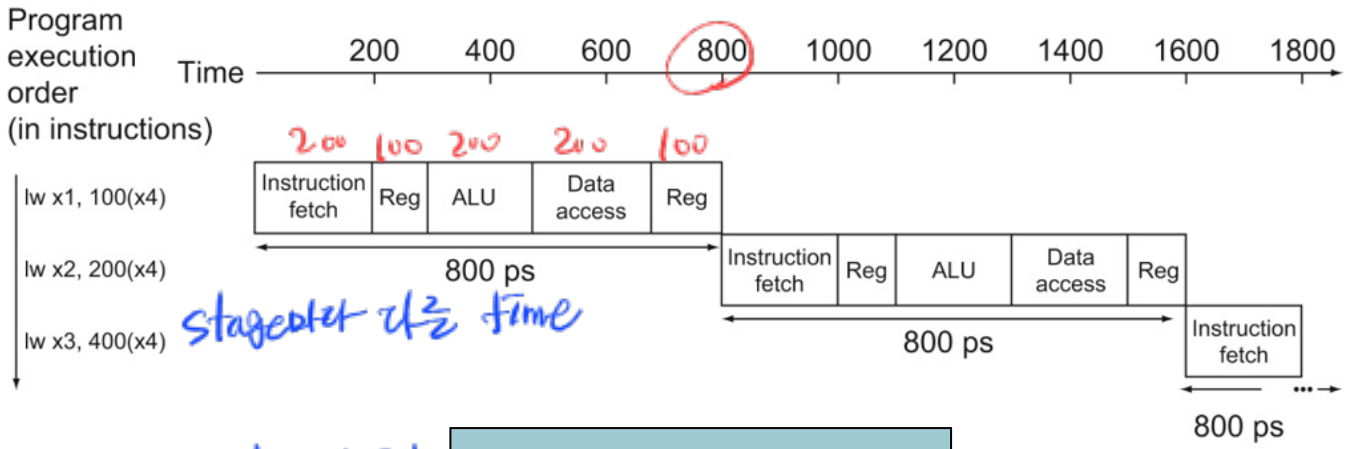
- Assume the time for each stages is
  - 100ps for register read or write
  - 200ps for other stages

Instruction type	Instruction fetch	Register read	ALU op	Memory access	Register write	Total time
`lw`	200ps	100 ps	200ps	200ps	100 ps	800ps
`sw`	200ps	100 ps	200ps	200ps		700ps
`R-format`	200ps	100 ps	200ps		100 ps	600ps
`beq`	200ps	100 ps	200ps			500ps

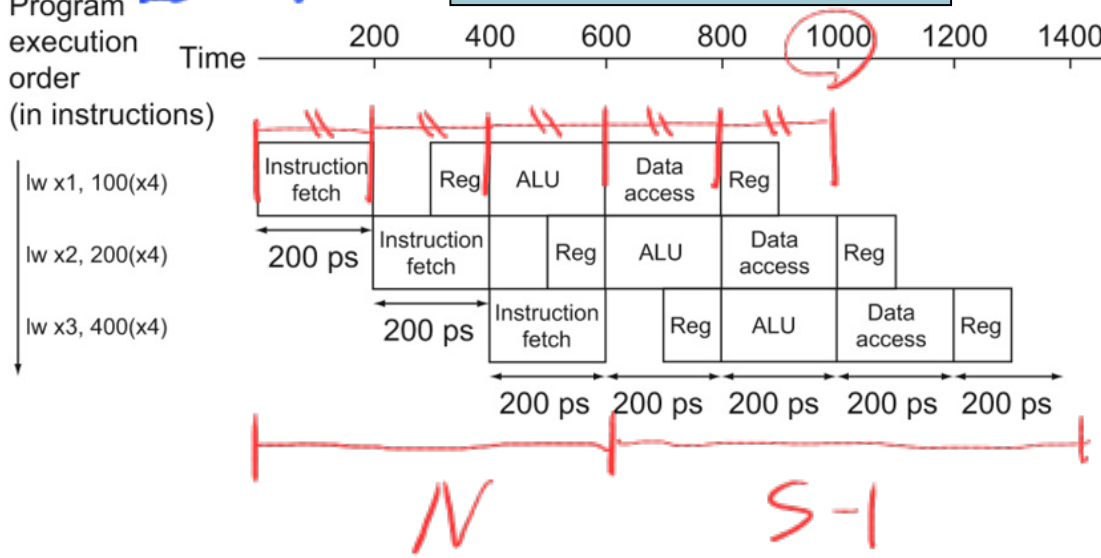
→ longest

Pipeline Performance

Single-cycle (T_c= 800ps)



Pipelined (T_c= 200ps)



2400/3 ins → 1400/3 ins

less than 2X (2배보다 작은 차이)

but instruction이 빨리끝남 → 차이 커짐

800 ps × # ins

200 ps × # ins + 800ps ≈ 4배 차이

↳ trivial

# of cycles in pipelining when # of stages is S and # of instructions is N

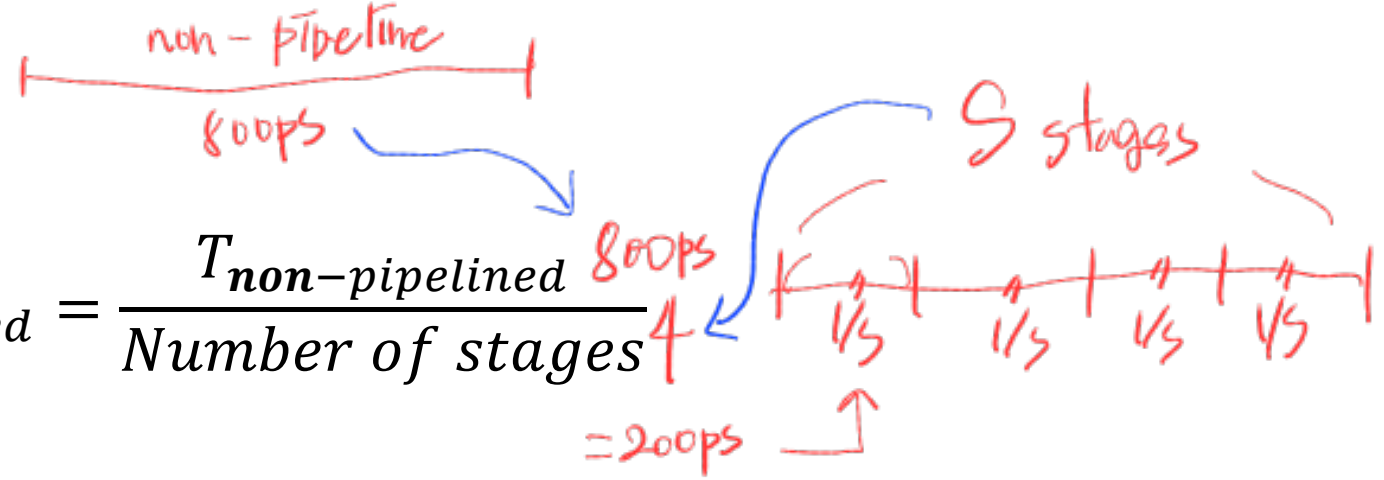
= (N + S − 1)

1000 ps − 200 ps

상수이므로 trivial

Pipeline Speedup

- If all stages are balanced
  - i.e., all take the same time



perfectly balanced pipeline에서만 가능

$$T_{pipelined} = \frac{T_{non-pipelined}}{Number\ of\ stages}$$

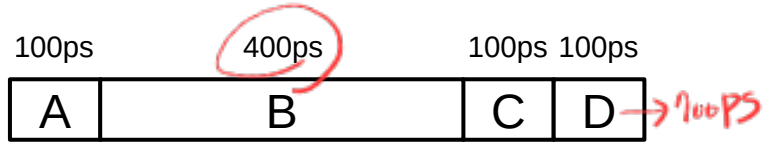
- If not balanced, speedup is limited
  - The longest stage determines the clock period

stage가 5개라면 → 160ps
but unbalance한 pipeline이면 → 200 ps
(200 100 200 200 100)

- Speedup due to increased throughput
  - Latency (time for each instruction) does not decrease

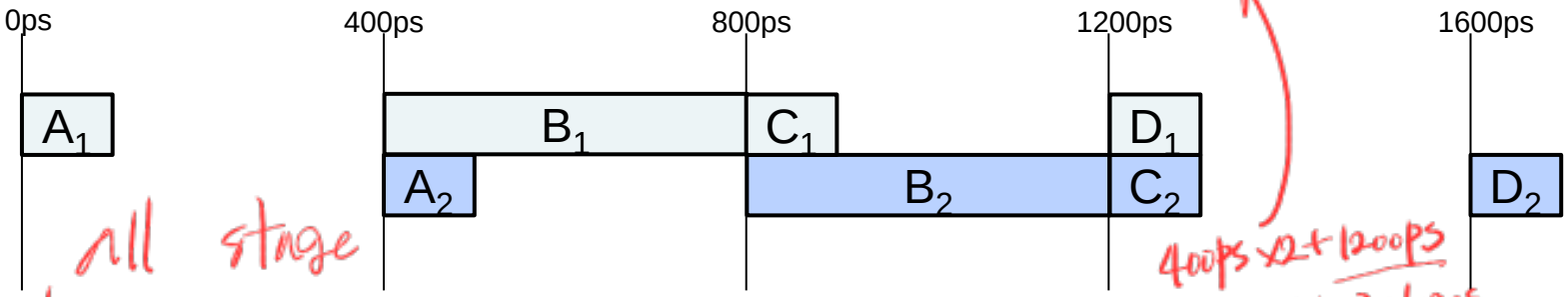
Latency가 아닌 throughput 증가

Pipeline Speedup Examples



non-pipelining → 700ps×2 = 1400ps/2inst

pipelining → 2000ps/2inst



all stage takes 400ps

400ps×2 + 1200ps

(4-1) stages

3 stages = 1200ps

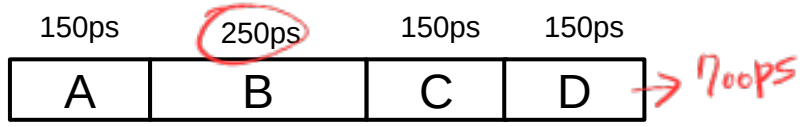
Instruction이 2개 pipelining이 효과가 나올 수 있는

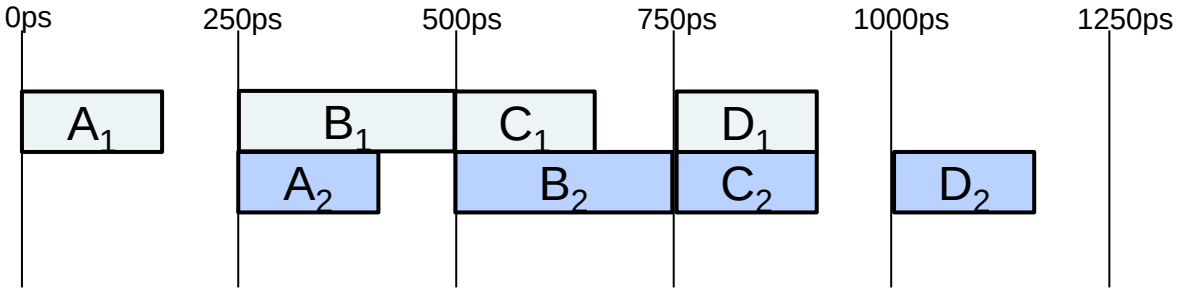
no pipelining

≈ 700ps/1inst

≈ 400ps/1inst

2개 Instruction이 만들어질 때...





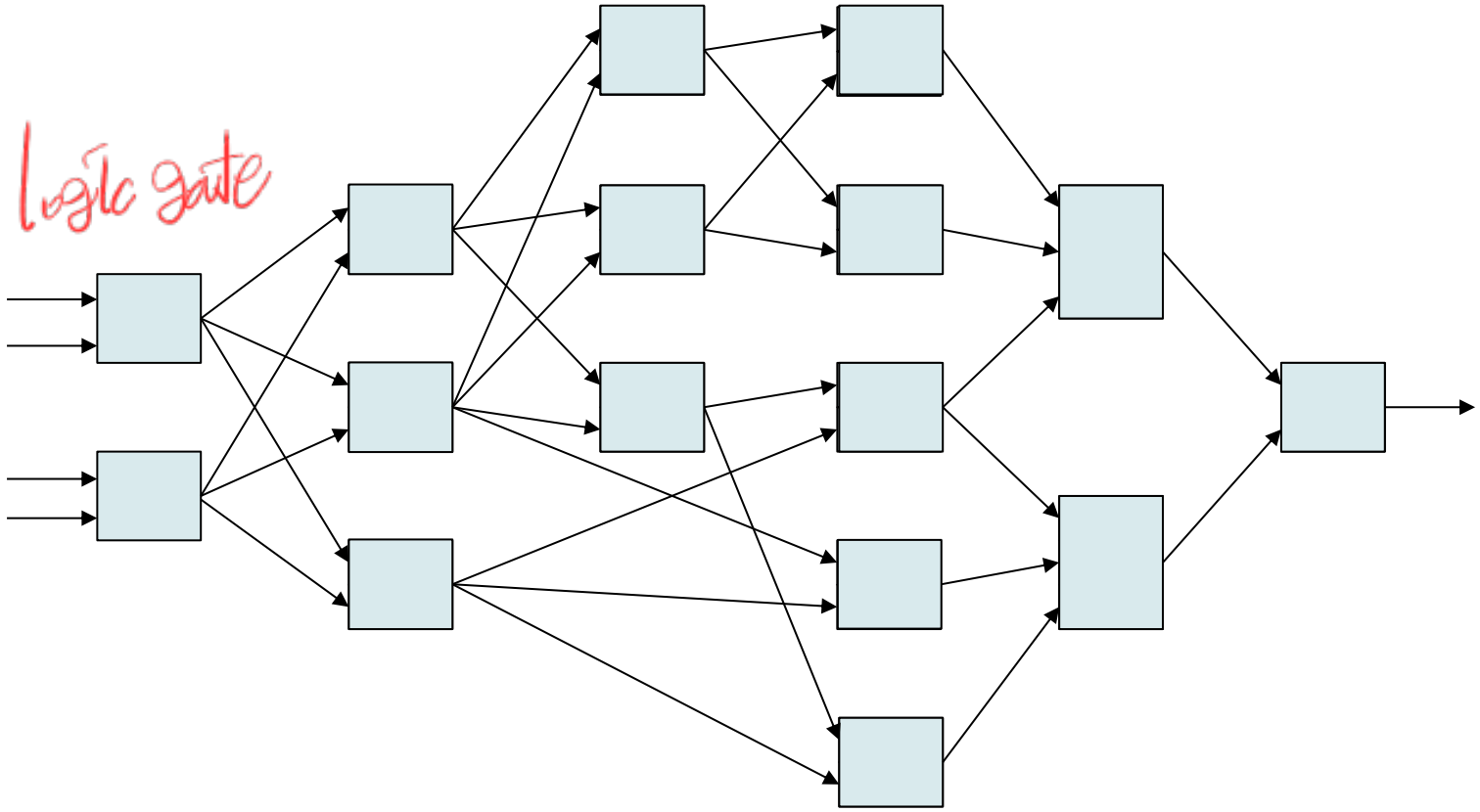
1250ps/2inst

≈ 250ps/1inst

좀더 balance한 pipelining

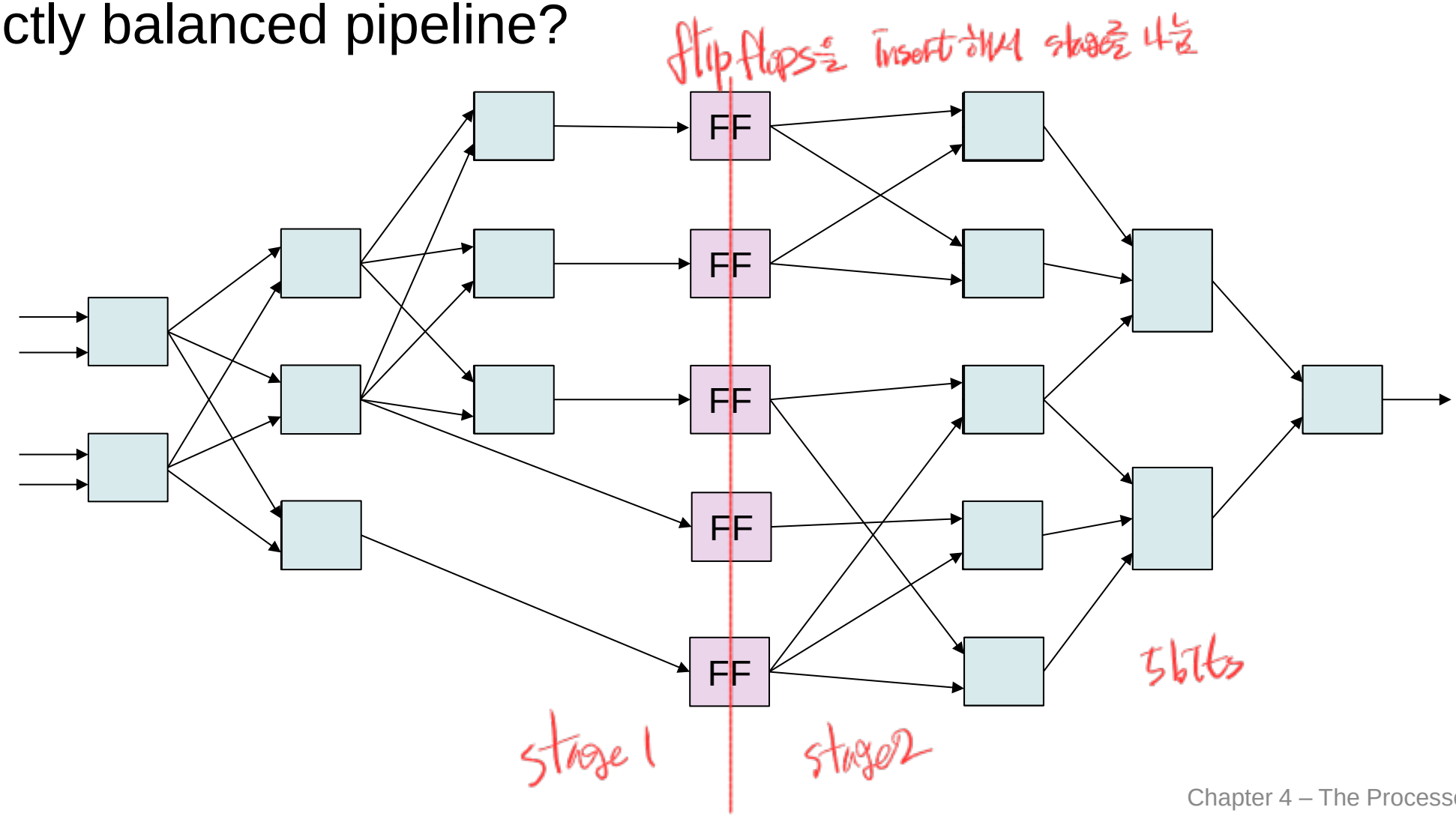
Pipeline Stage

- Can we split a combinational logic at arbitrary locations to obtain a perfectly balanced pipeline?



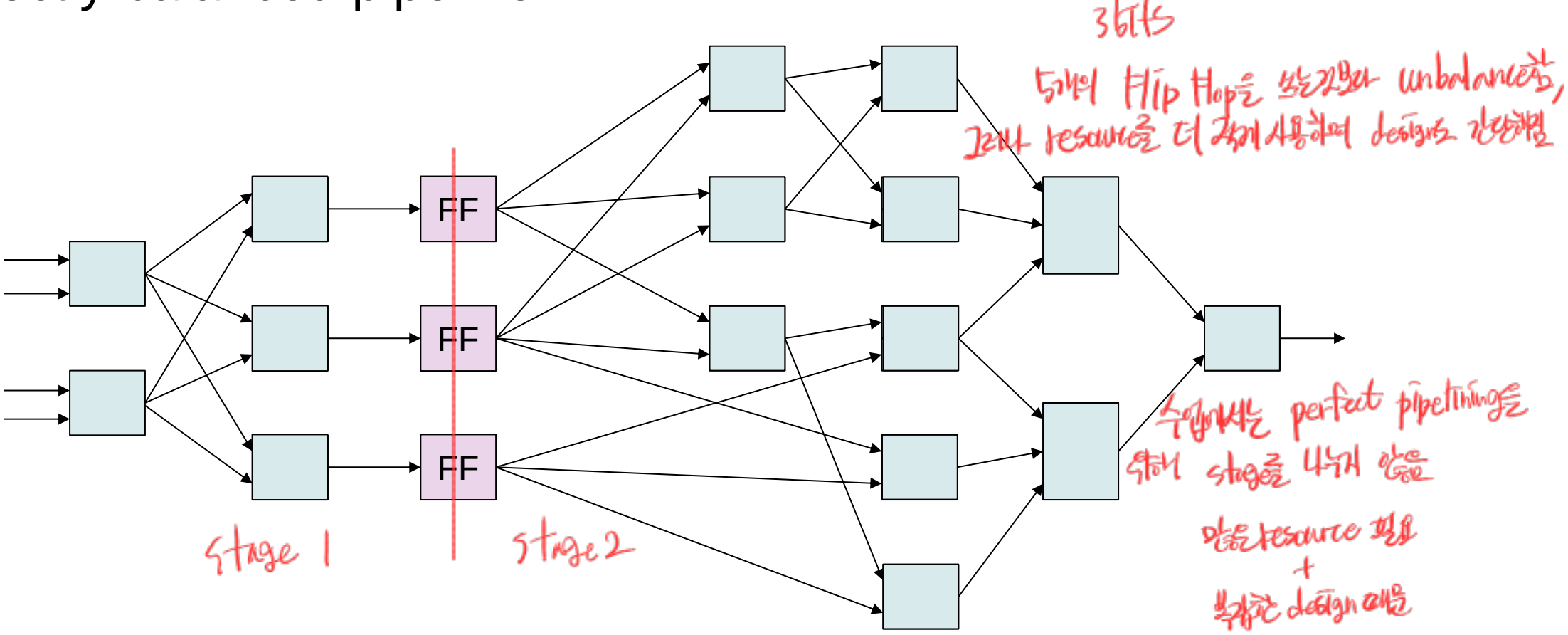
Pipeline Stage

- Can we split a combinational logic at arbitrary locations to obtain a perfectly balanced pipeline?



Pipeline Stage

- Can we split a combinational logic at arbitrary locations to obtain a perfectly balanced pipeline?



Pipelining and ISA Design

Instruction set Architecture

- RISC-V ISA designed for pipelining

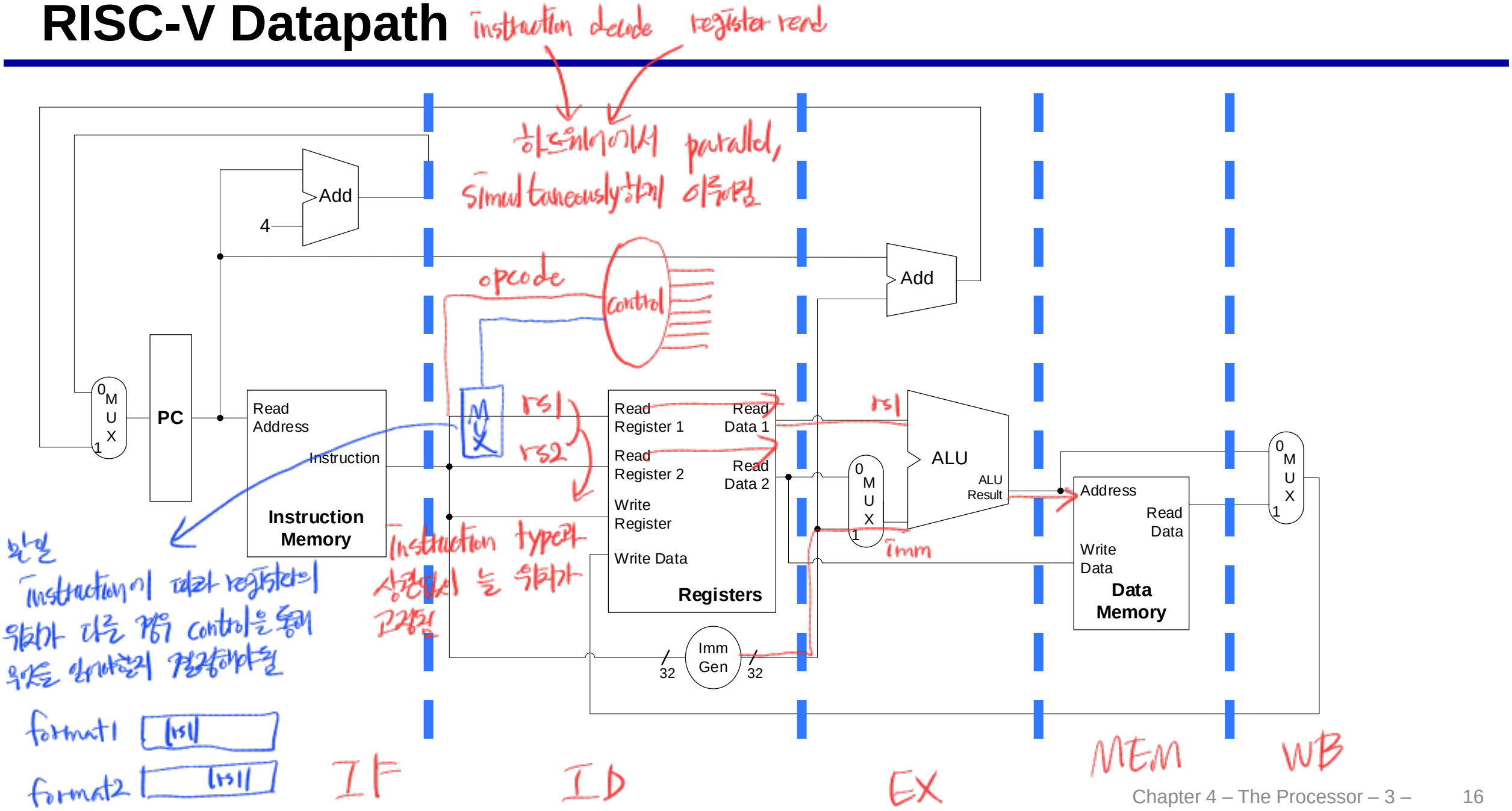
  x86 ISA

  - All instructions are 32-bits (4bytes) 이하의 크기 ⟷ 1~15 byte 내의 instruction이 될 byte수를 알 수 없음
    - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to ~~17~~ 15-byte instructions
  - Few and regular instruction formats ⟷ many, complex instruction format
    - Can decode and read registers in one step rs1, rs2
  - Load/store addressing ⟷ complex memory addressing modes
    - Can calculate address in 3rd stage, access memory in 4th stage

    EX

    *rs1 + imm

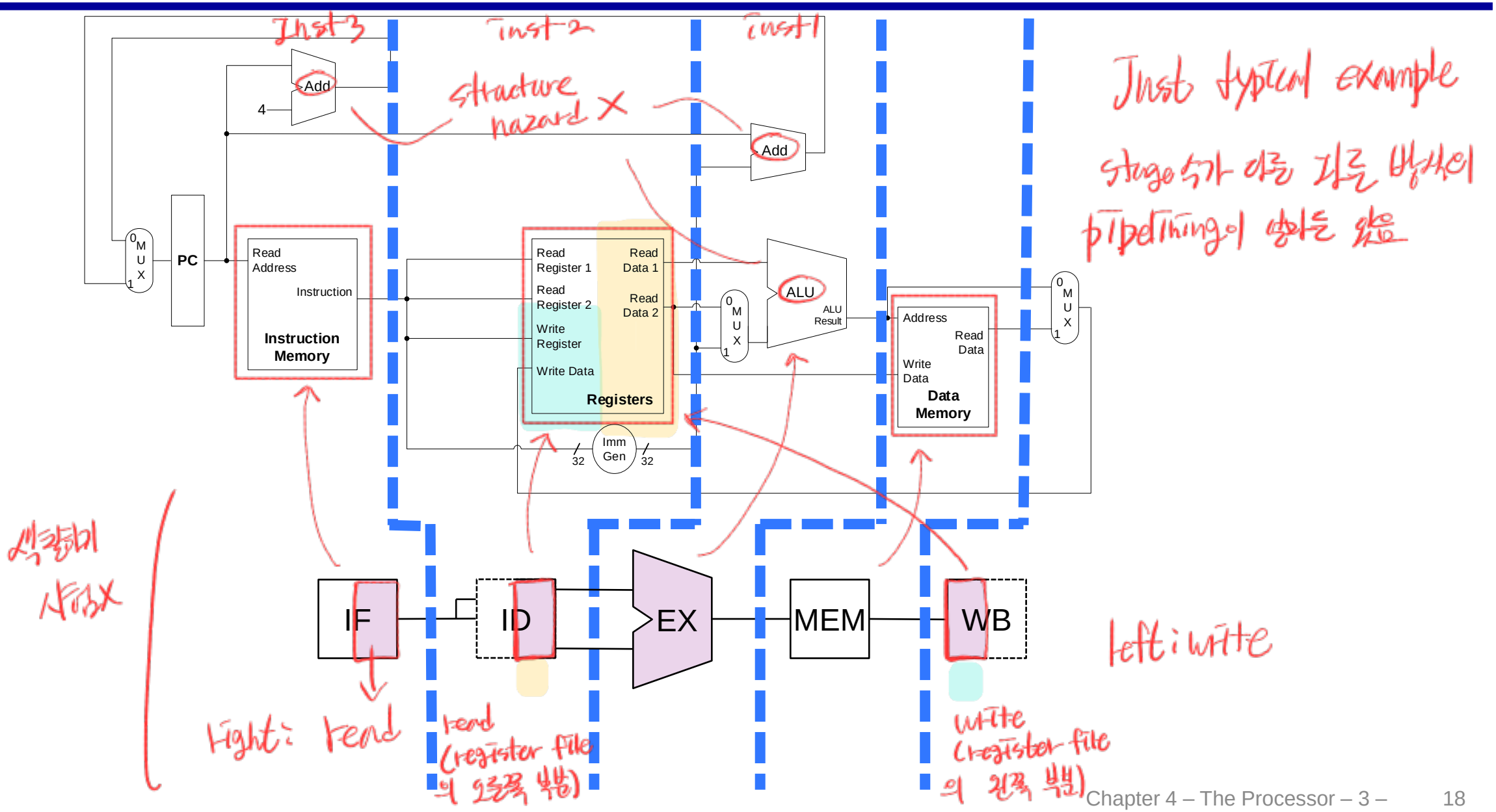
RISC-V Datapath



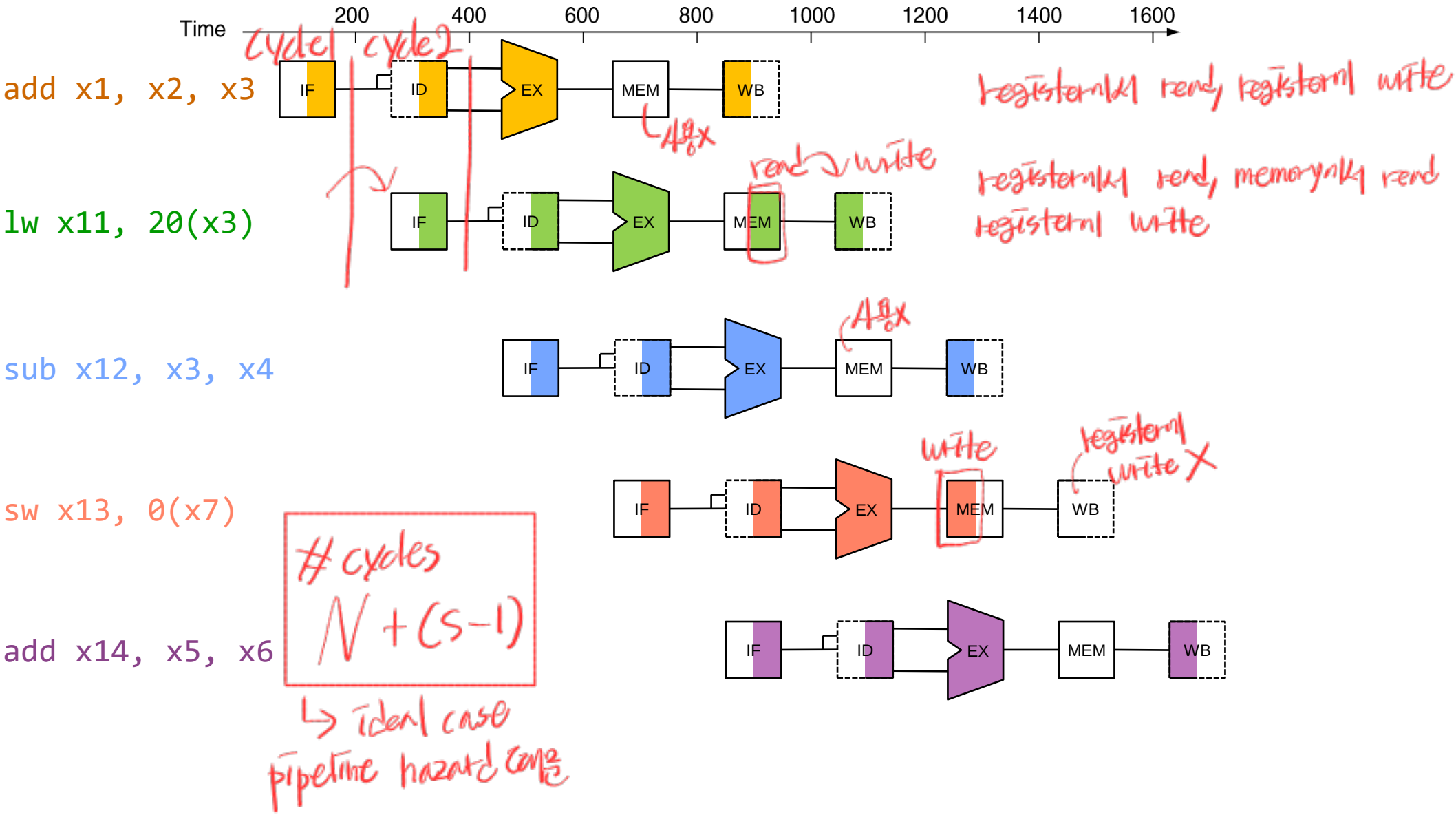
RISC-V Pipeline Stages

- Five stages, one step per stage
  1. **IF**: Instruction fetch from memory
  2. **ID**: Instruction decode & register read
  3. **EX**: Execute operation or calculate address
  4. **MEM**: Access memory operand
  5. **WB**: Write result back to register

Simplified Representation of RISC-V Pipeline



Simplified Representation of RISC-V Pipeline



Simplified Representation of RISC-V Pipeline

later

1st `add x1, x2, x3`

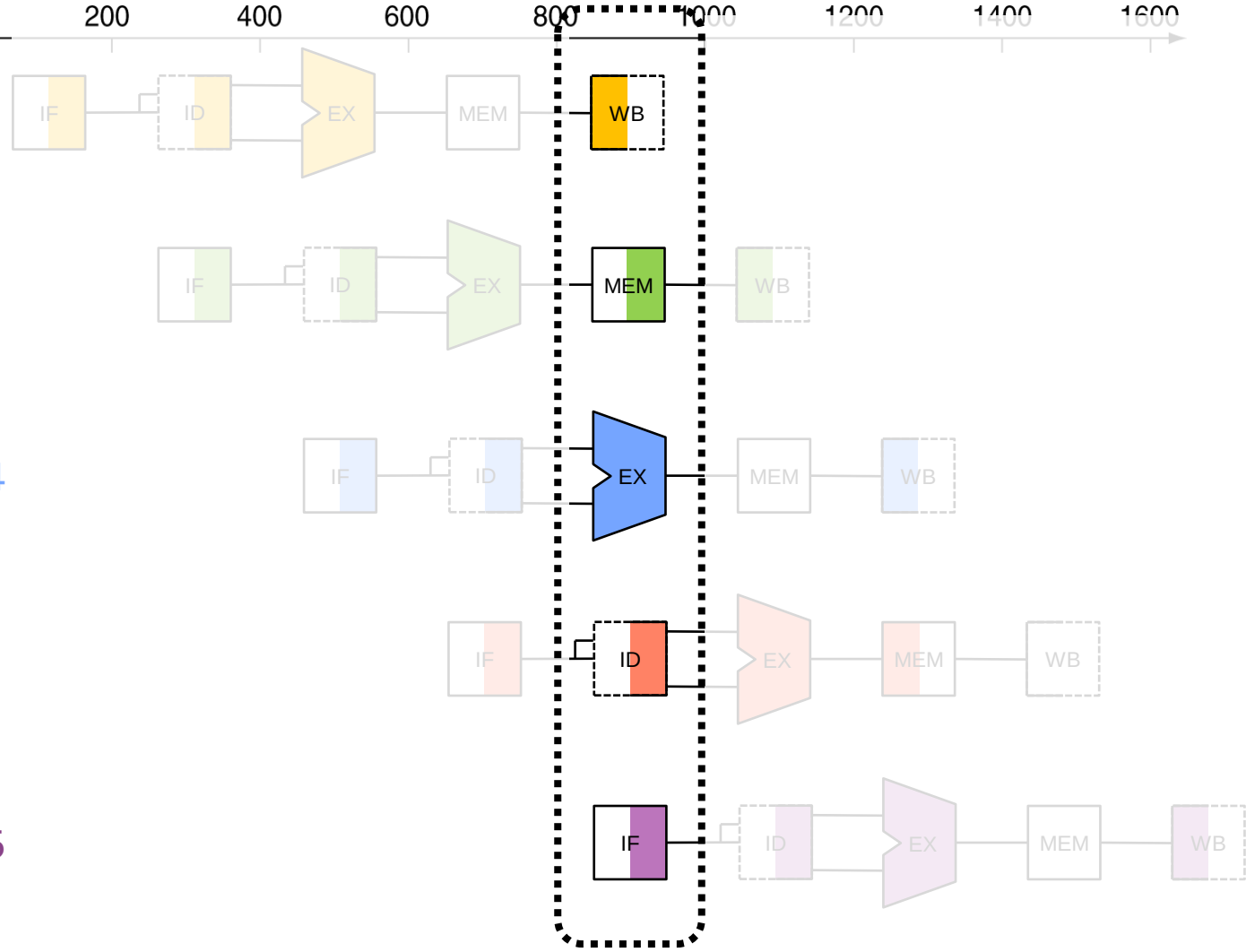
2nd `lw x11, 20(x3)`

3rd `sub x12, x3, x4`

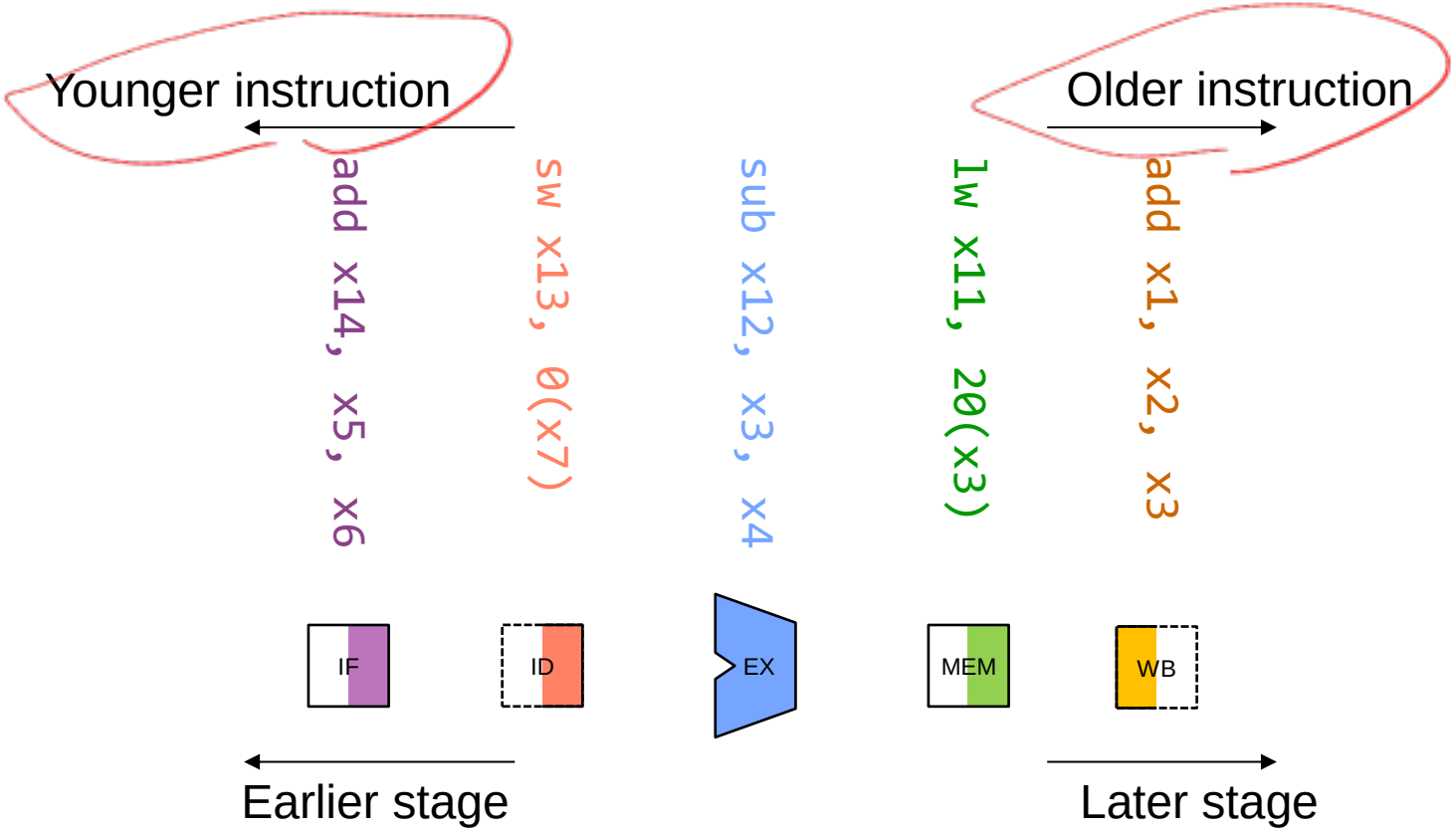
4th `sw x13, 0(x7)`

5th `add x14, x5, x6`

earlier



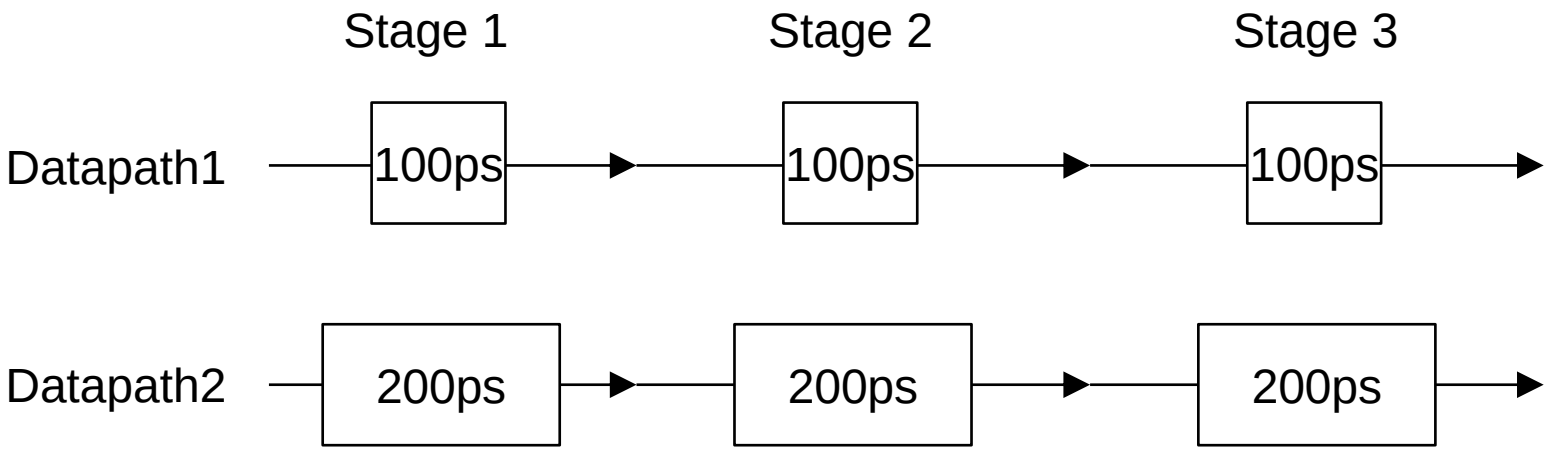
Instructions in a Pipeline



Dividing Pipeline Stages

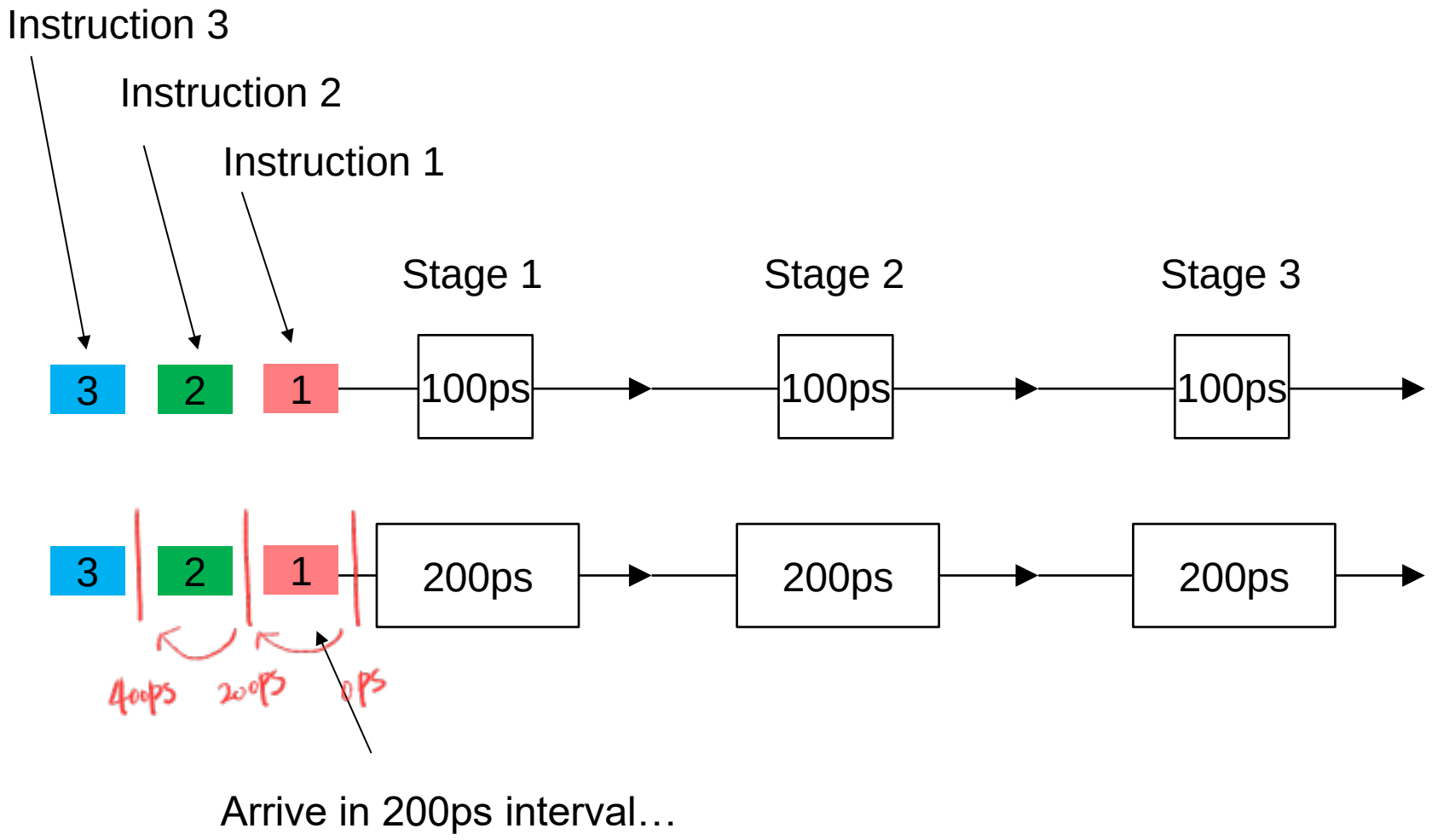
- Need to synchronize multiple datapaths

propagation delay

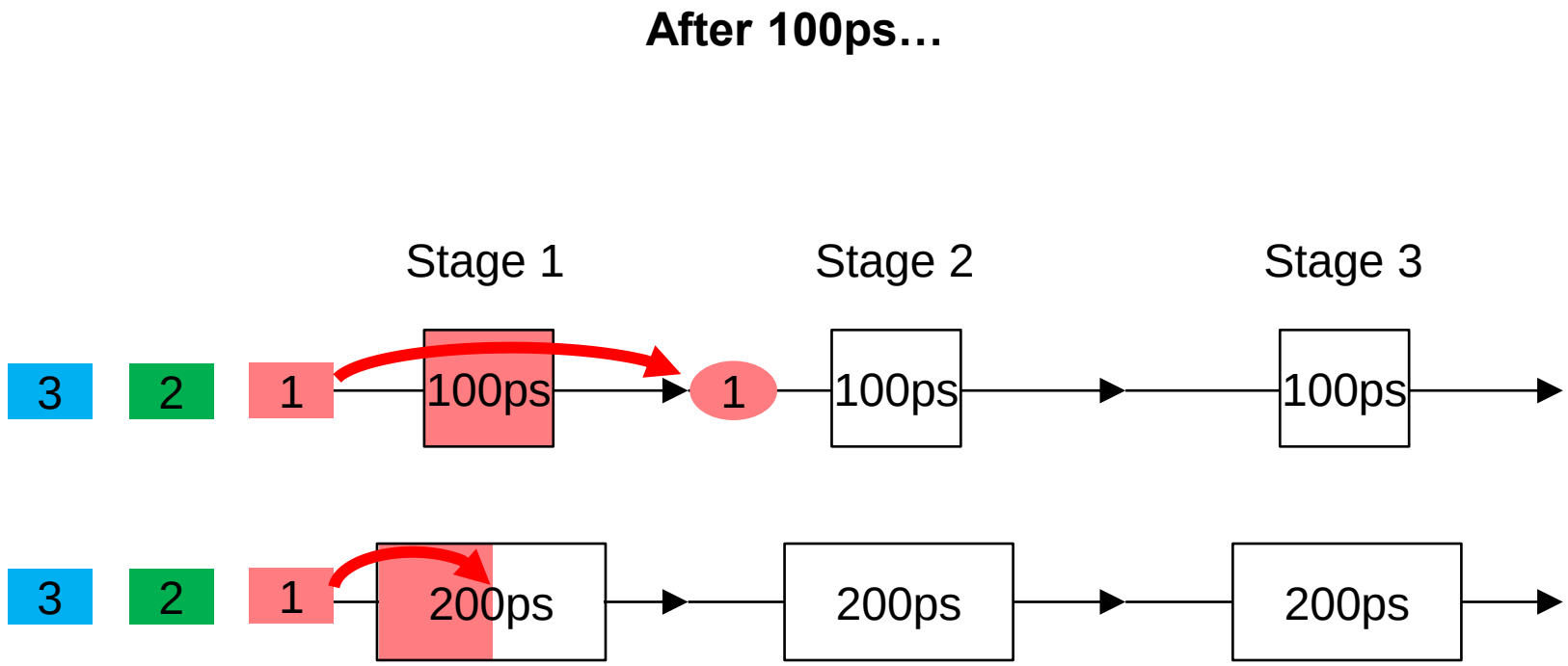


- (Almost) impossible to do it with combinational circuits only

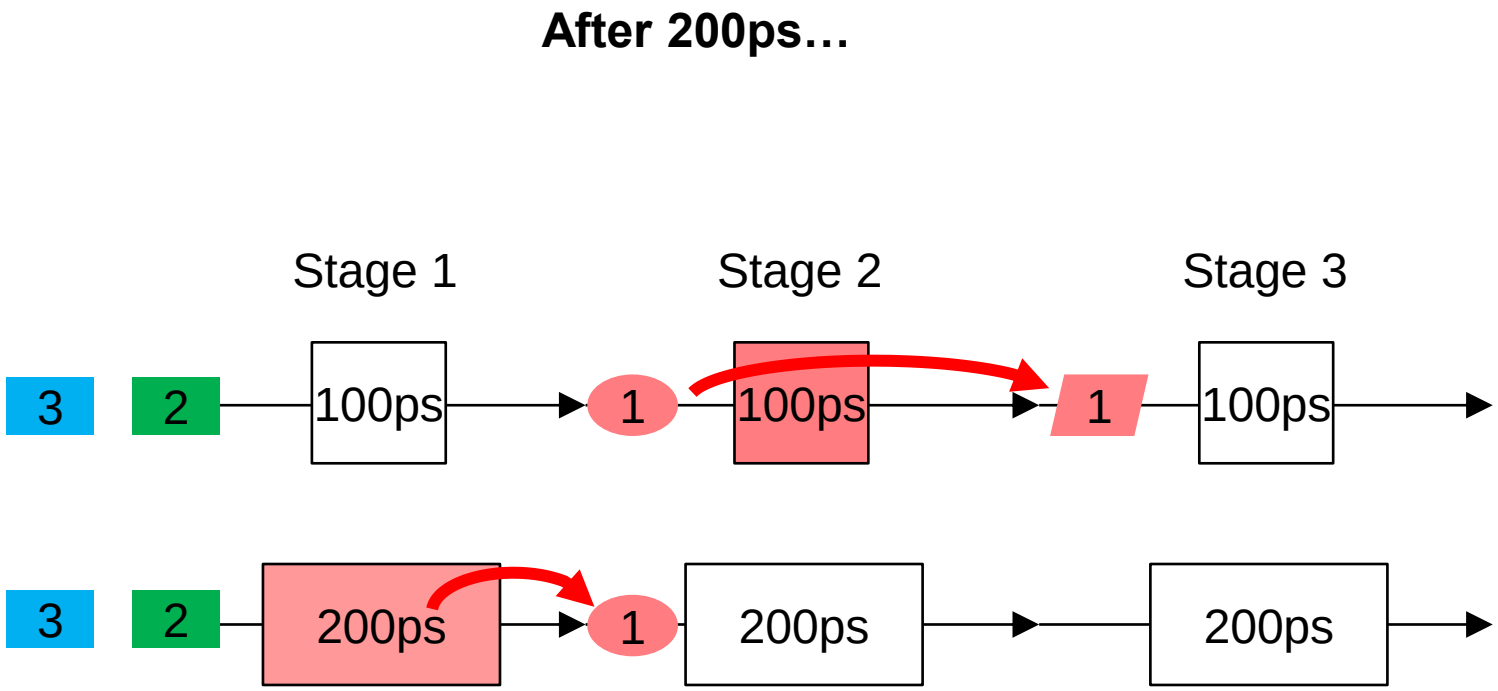
No separation?



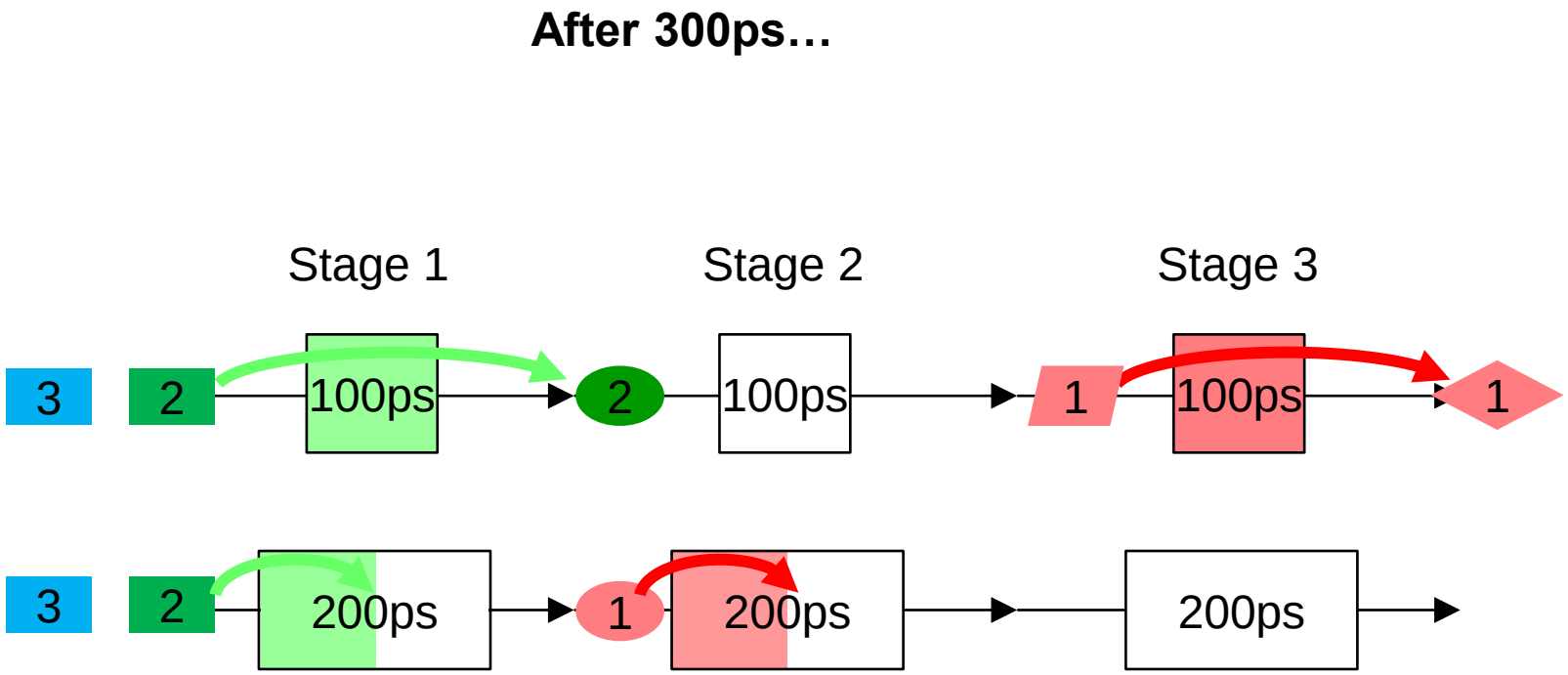
No separation?



No separation?



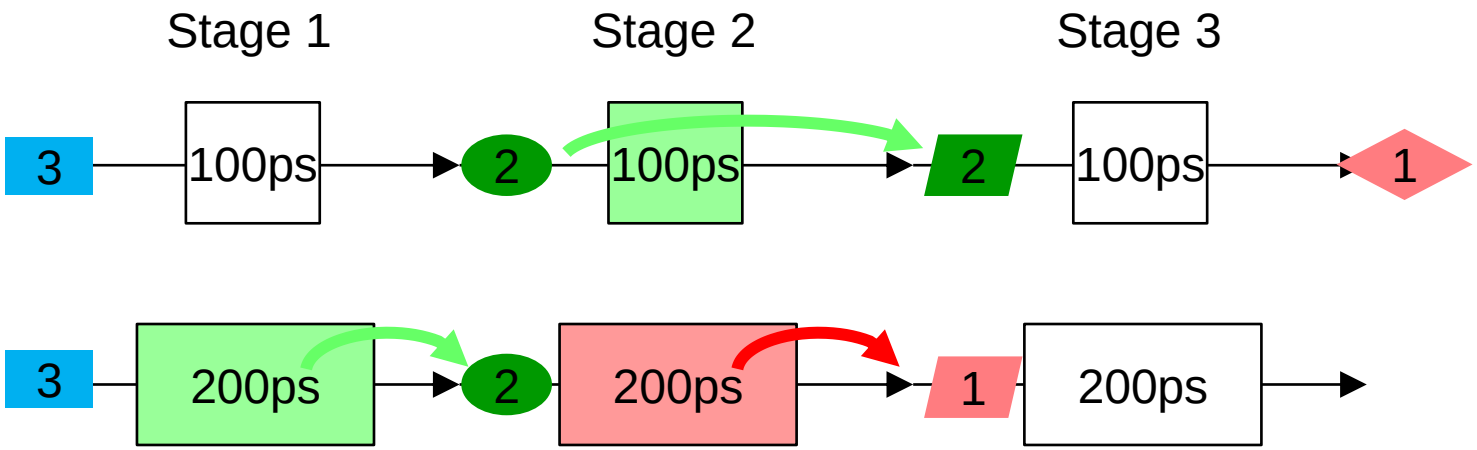
No separation?



Instructions in stage 2 do not match!

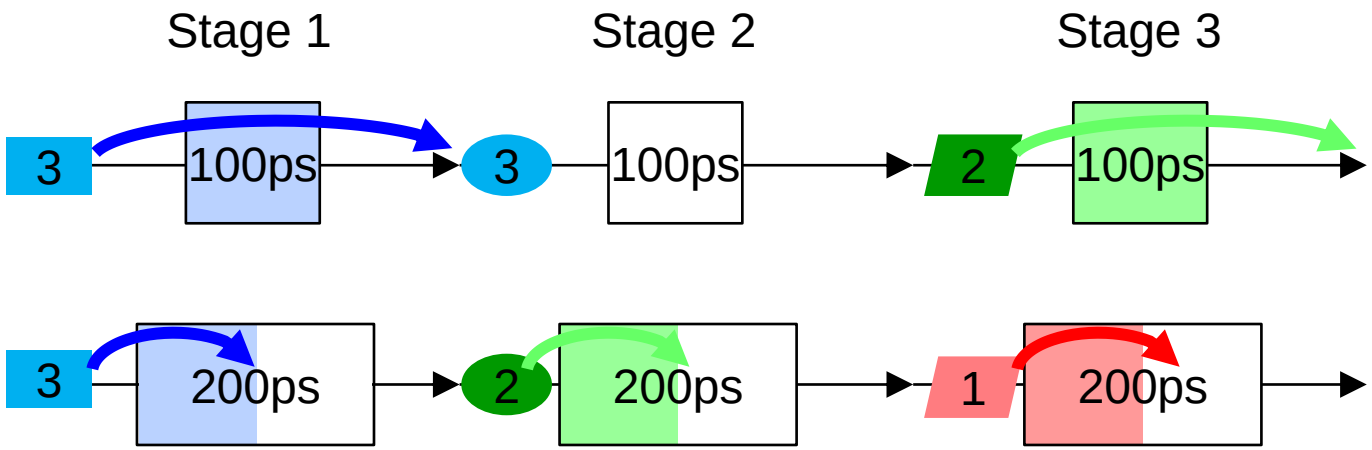
No separation?

After 400ps…

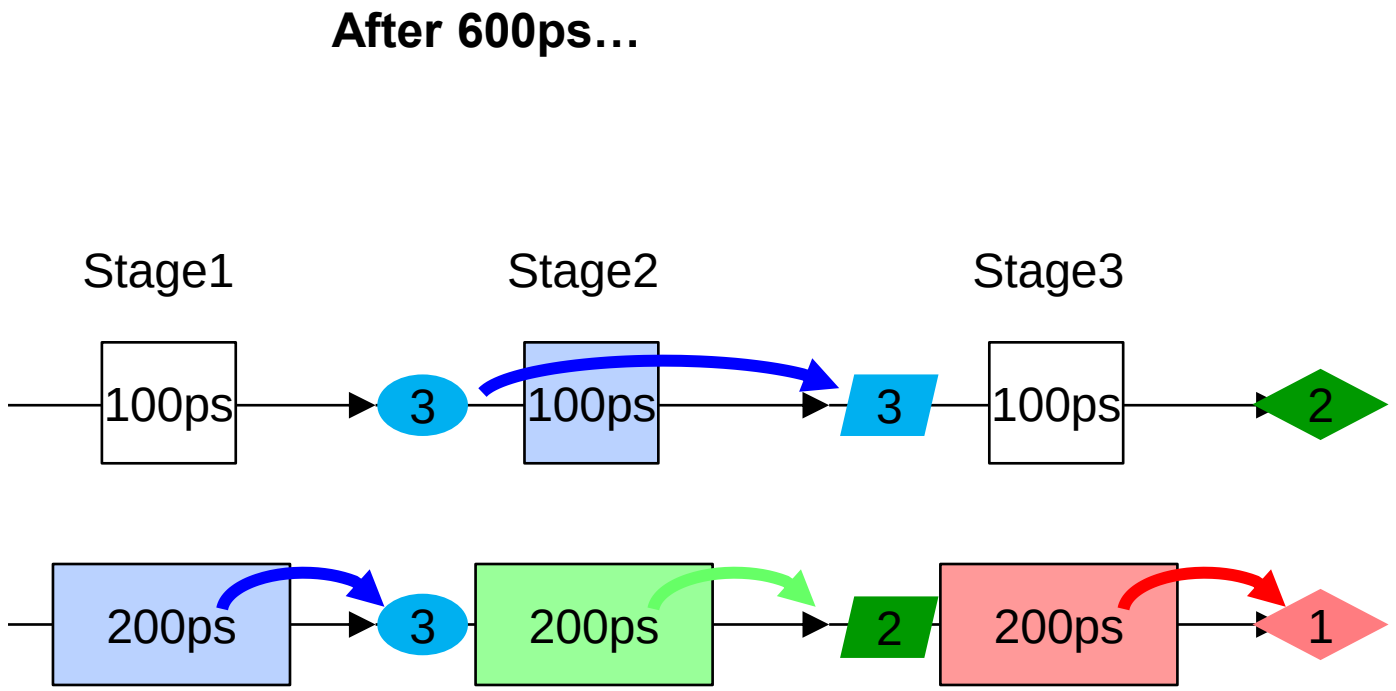


No separation?

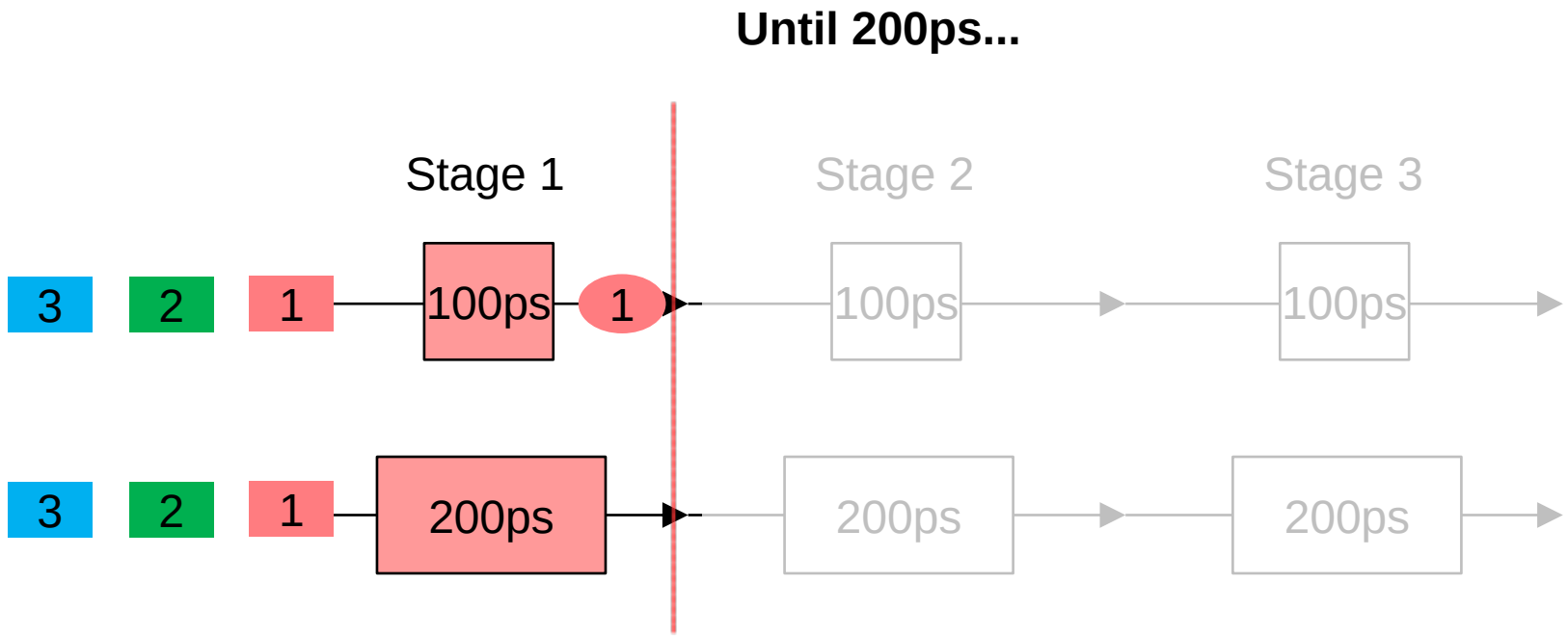
After 500ps…



No separation?

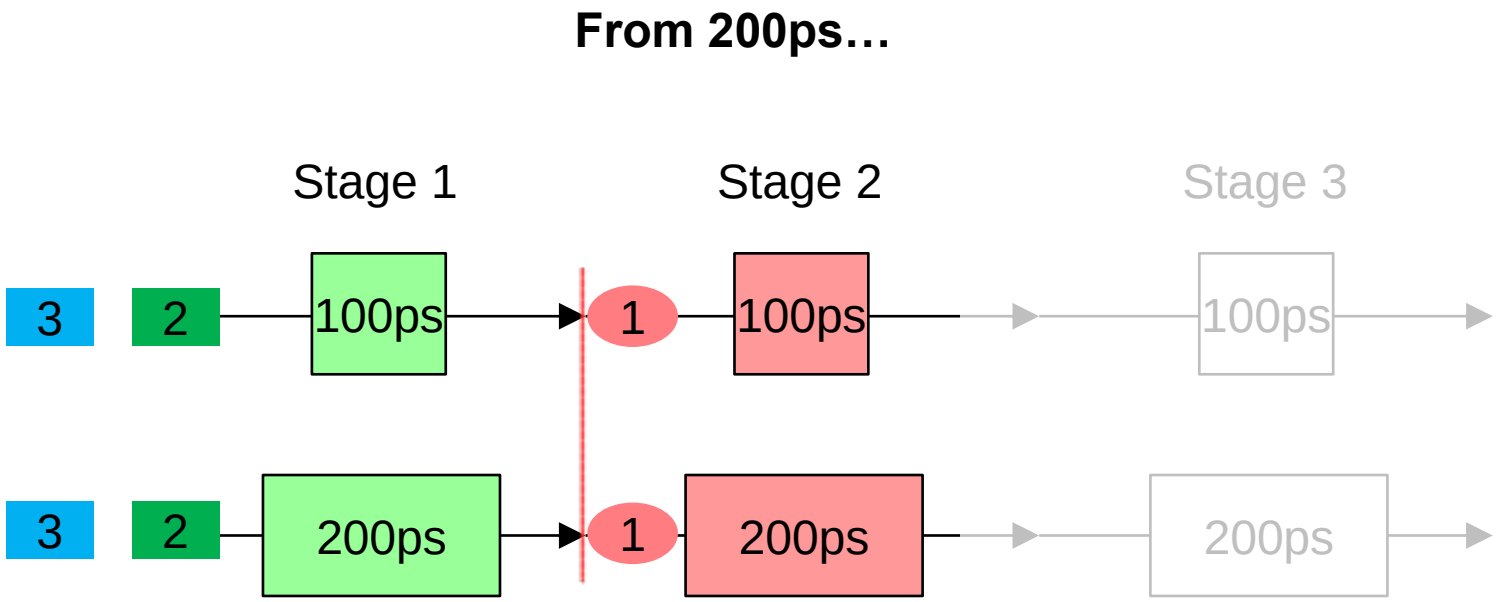


Dividing Stages – What should be done…



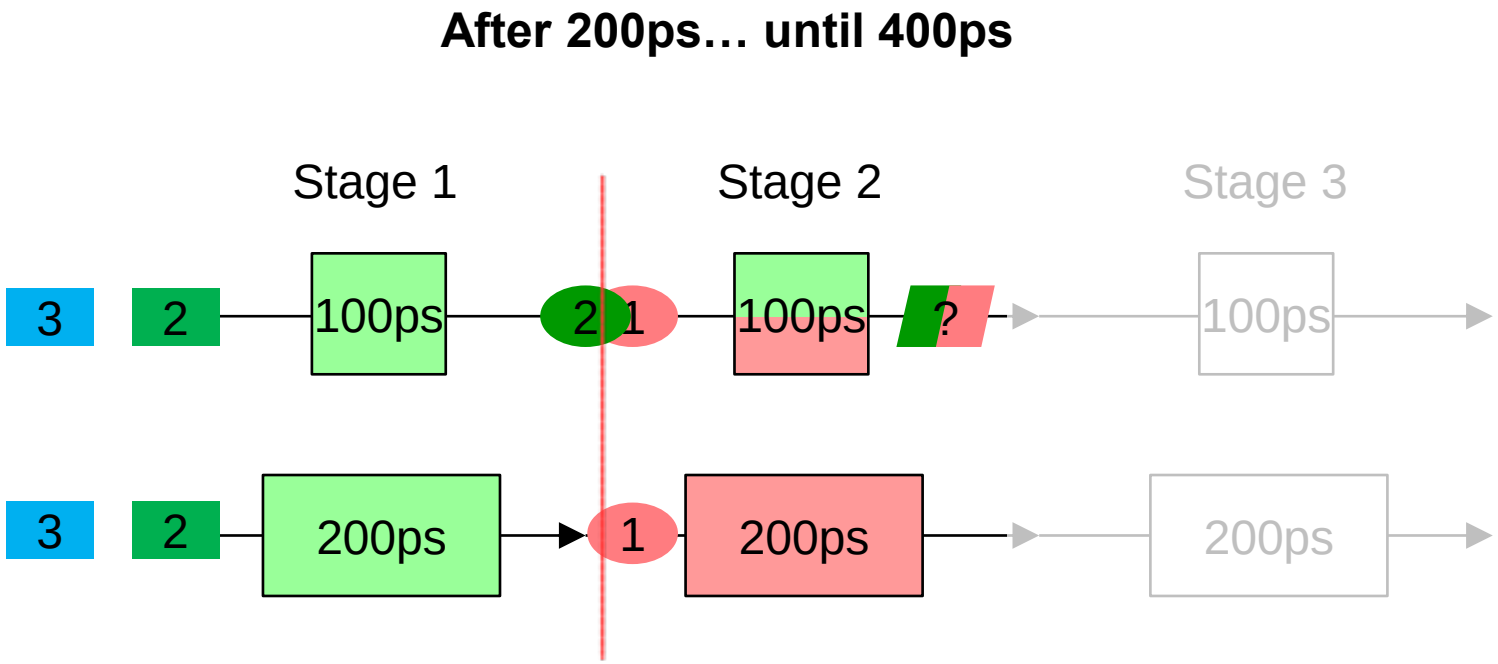
- Do not start stage2 yet…
- Wait until datapath2 also completes

Dividing Stages – What should be done…



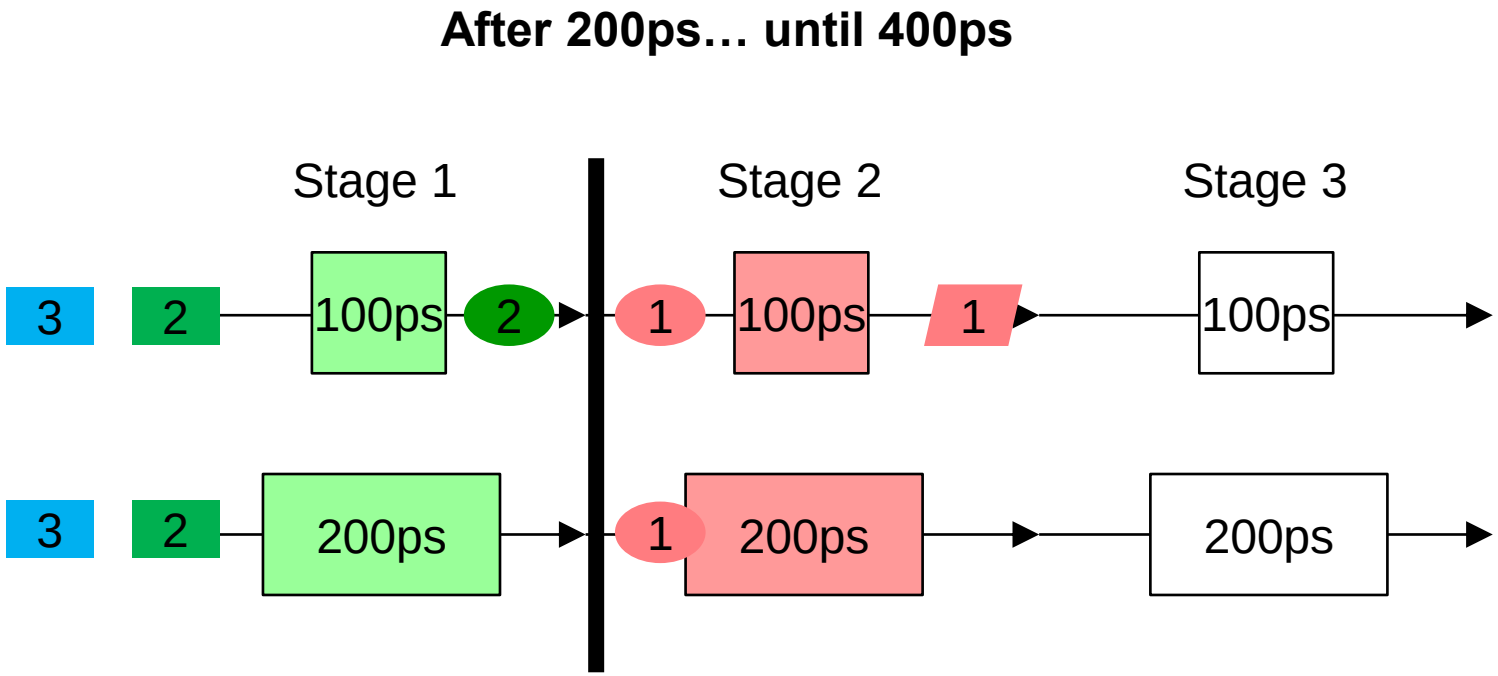
- Drive stage2 (i.e., give input data to stage2)

Dividing Stages – What should be done…



- Stage1 produces new output for Instruction2
- However, we still need to drive stage2 with Instruction1

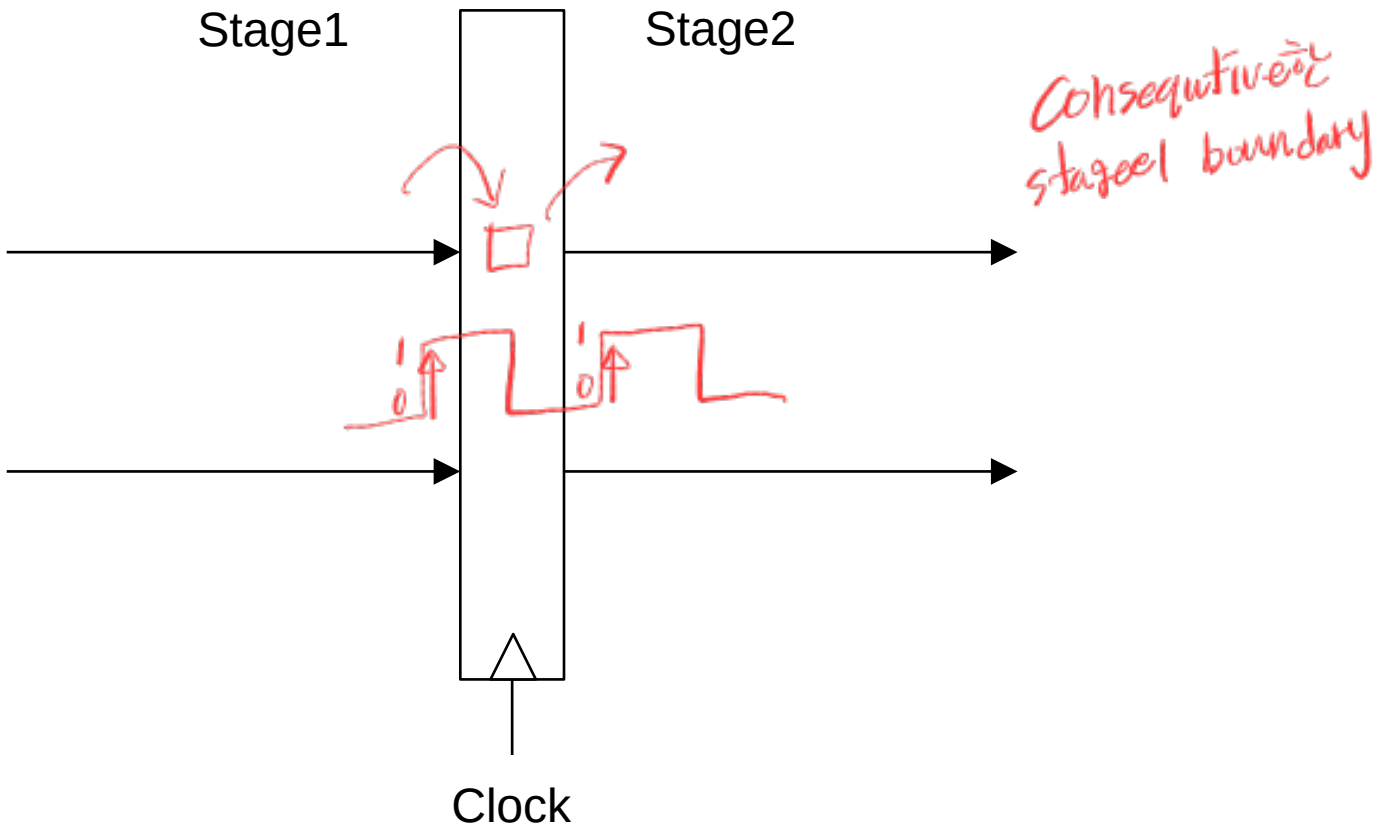
Dividing Stages – What should be done…



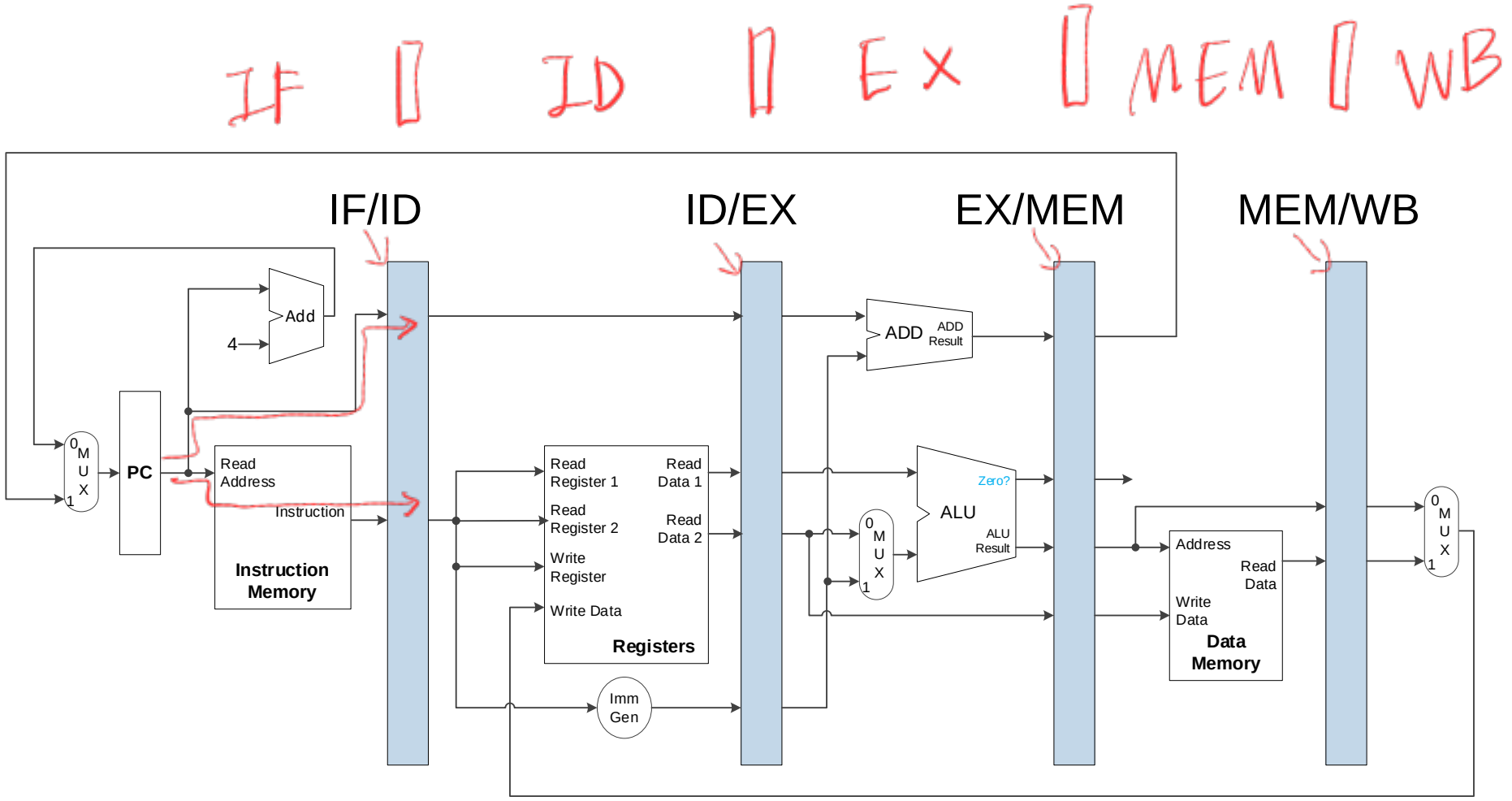
Between stages, we need a "**barrier**" that remembers the previous instruction and hold the next instruction

Using Registers

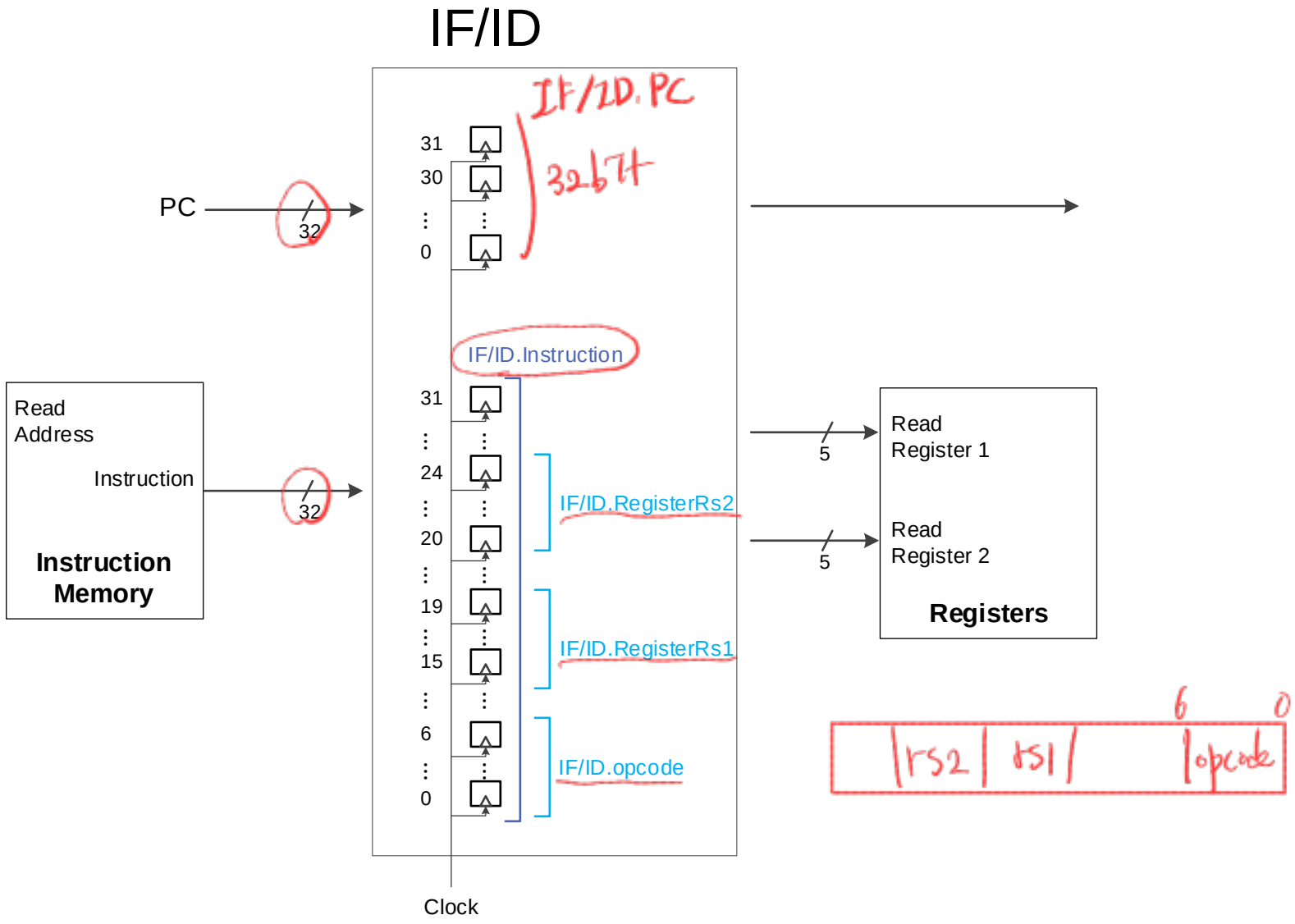
- Need registers (flip-flops) between stages



Pipeline Registers

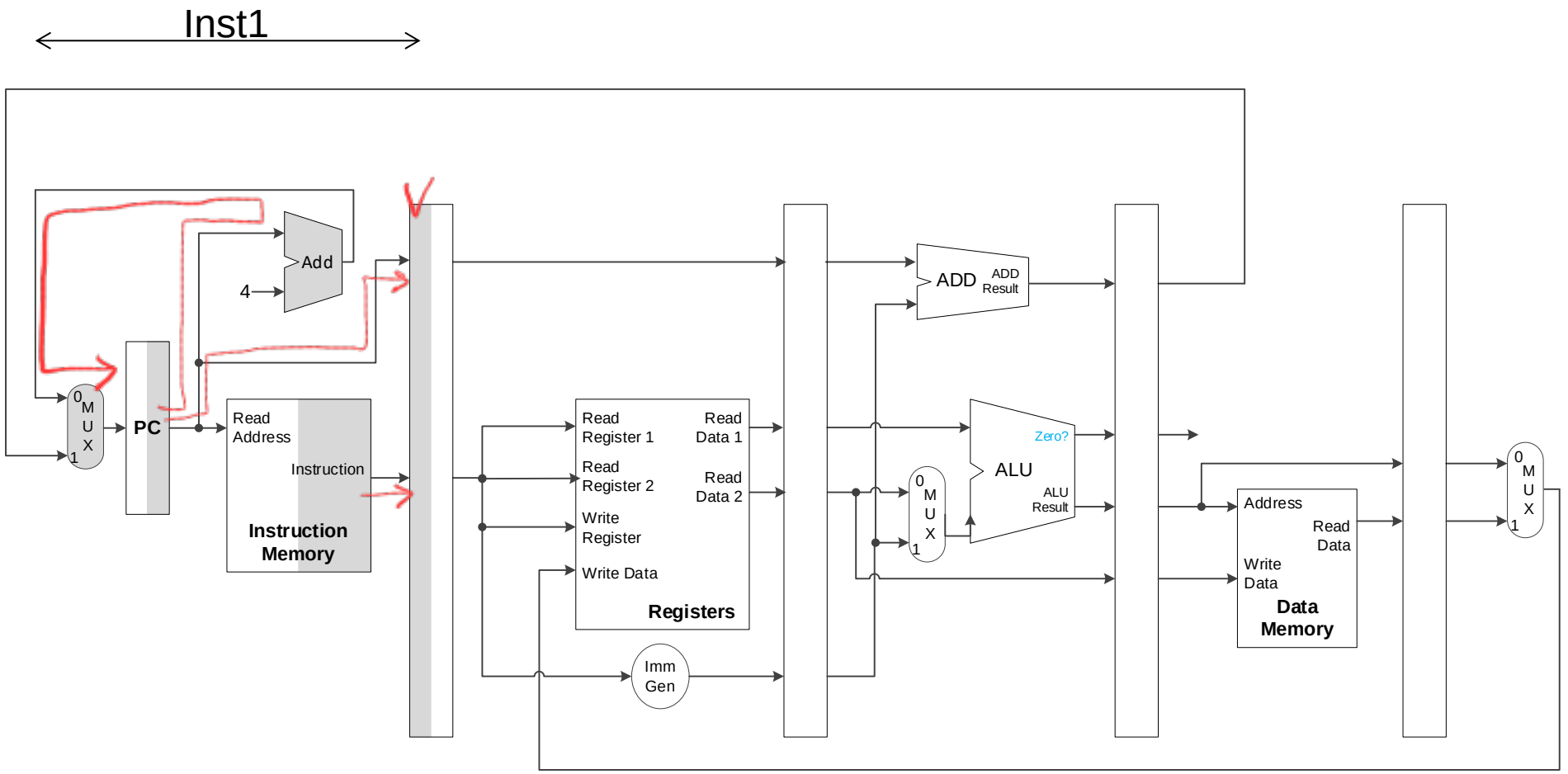


Pipeline Register Details



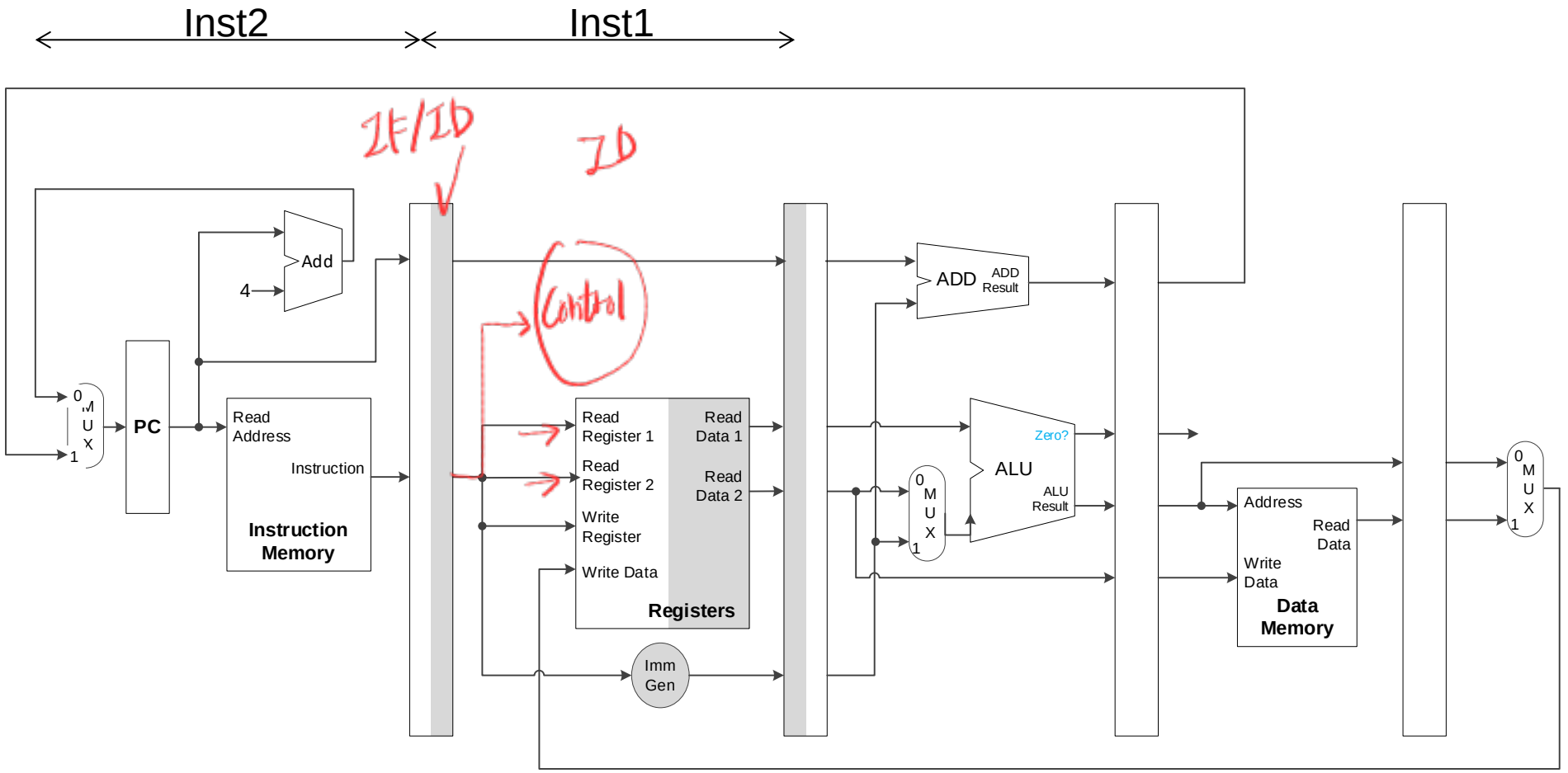
IF stage

cycle1



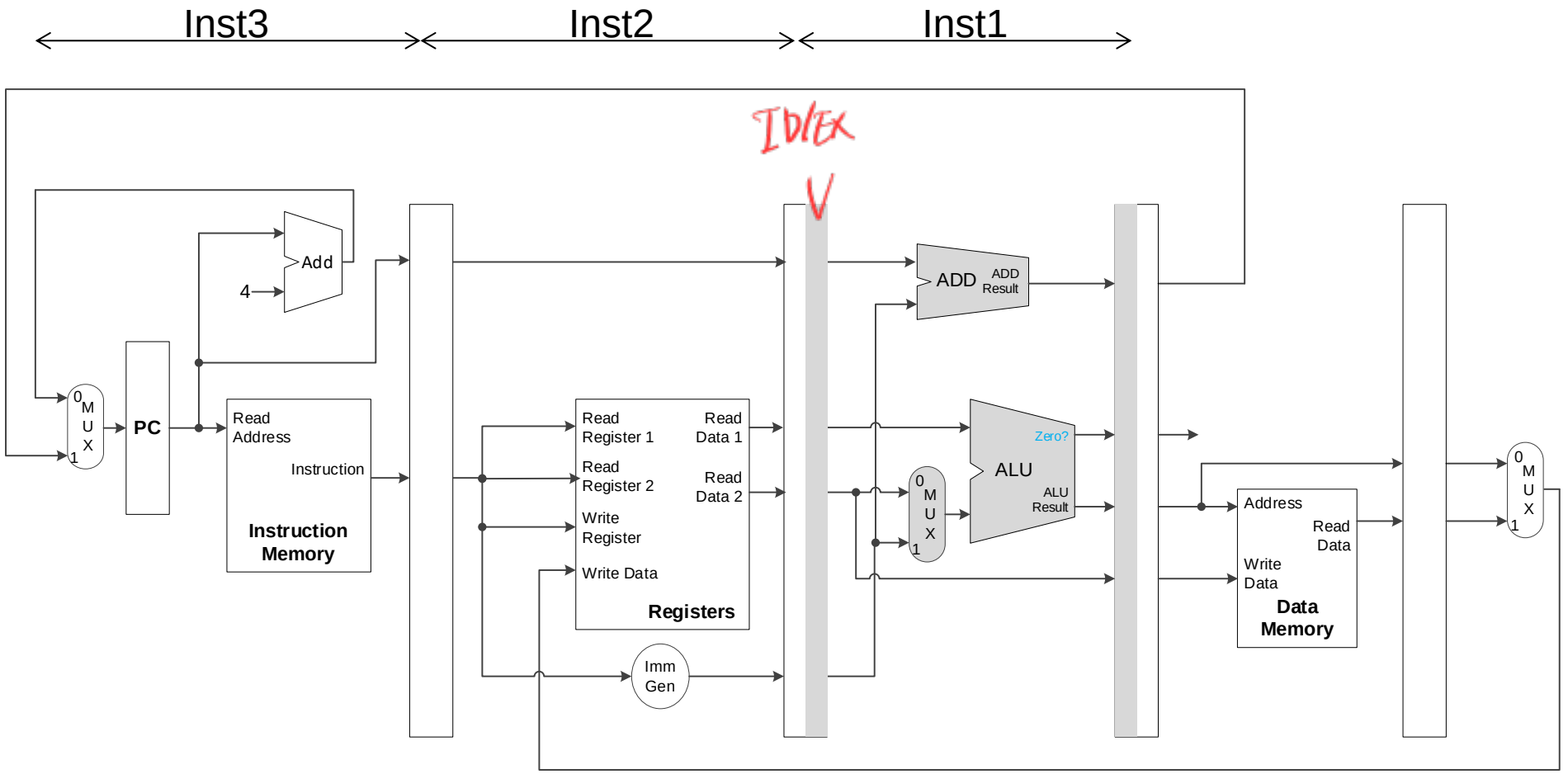
ID stage

cycle2



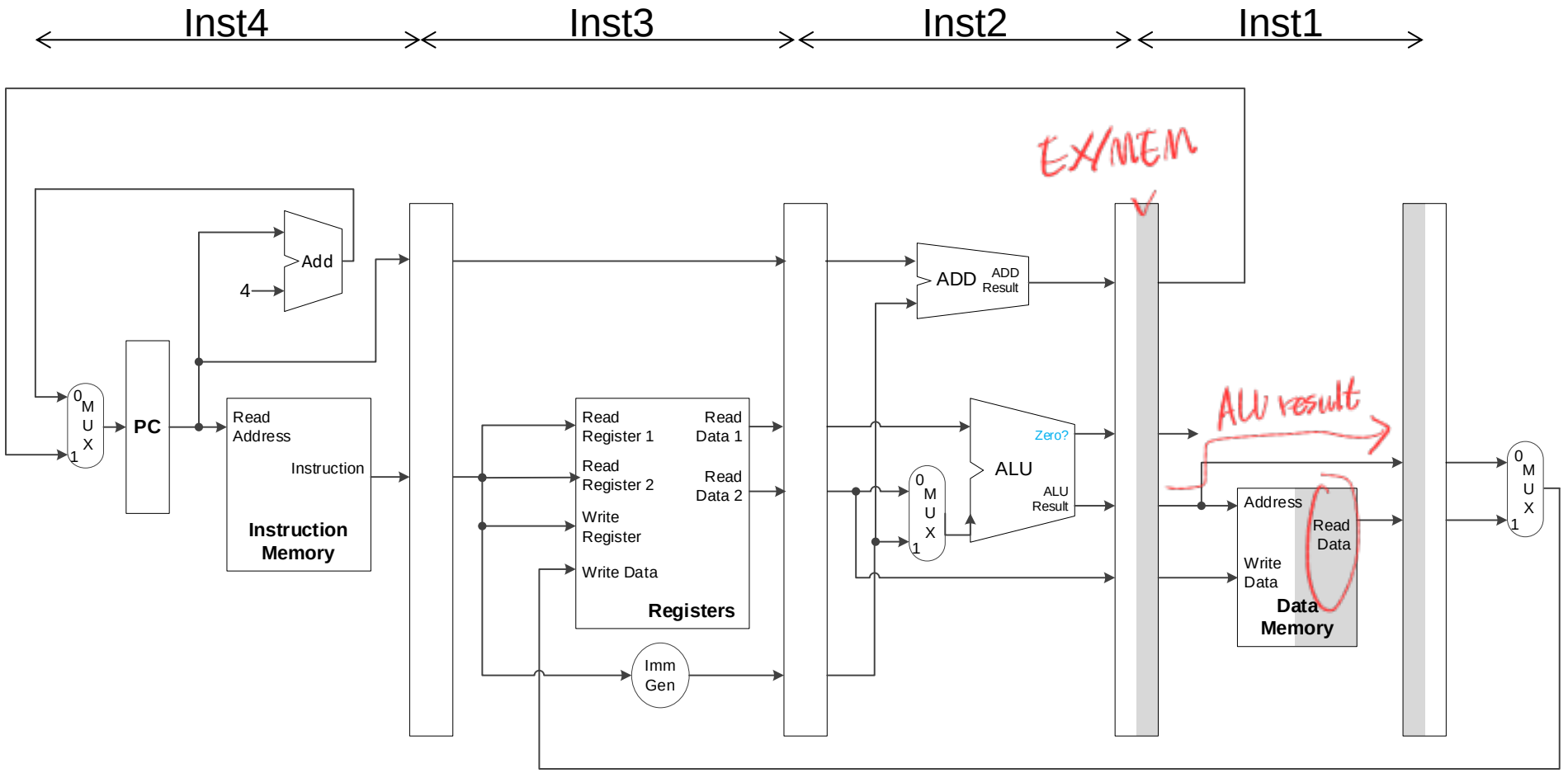
EX stage

cycle3

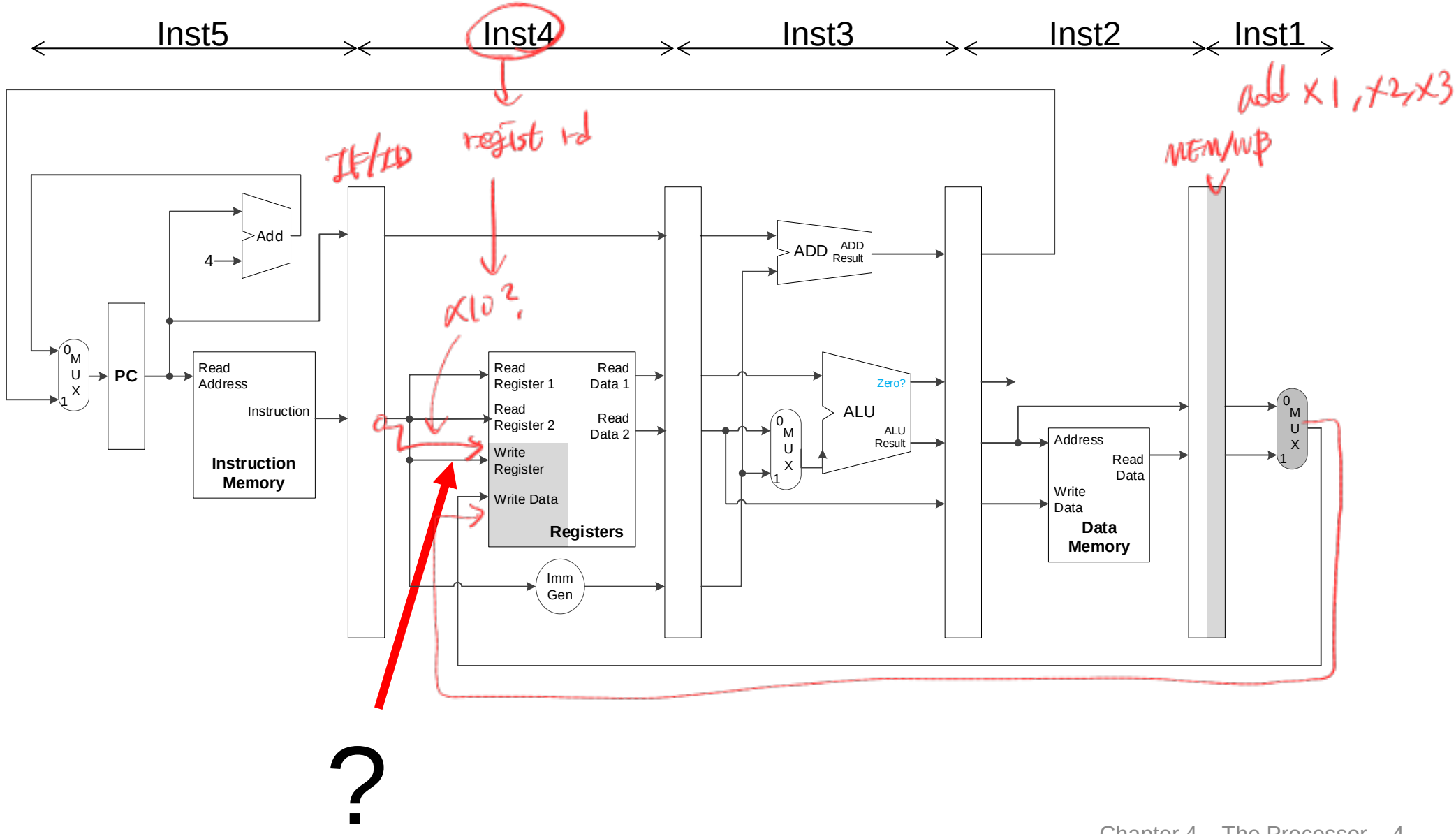


MEM stage

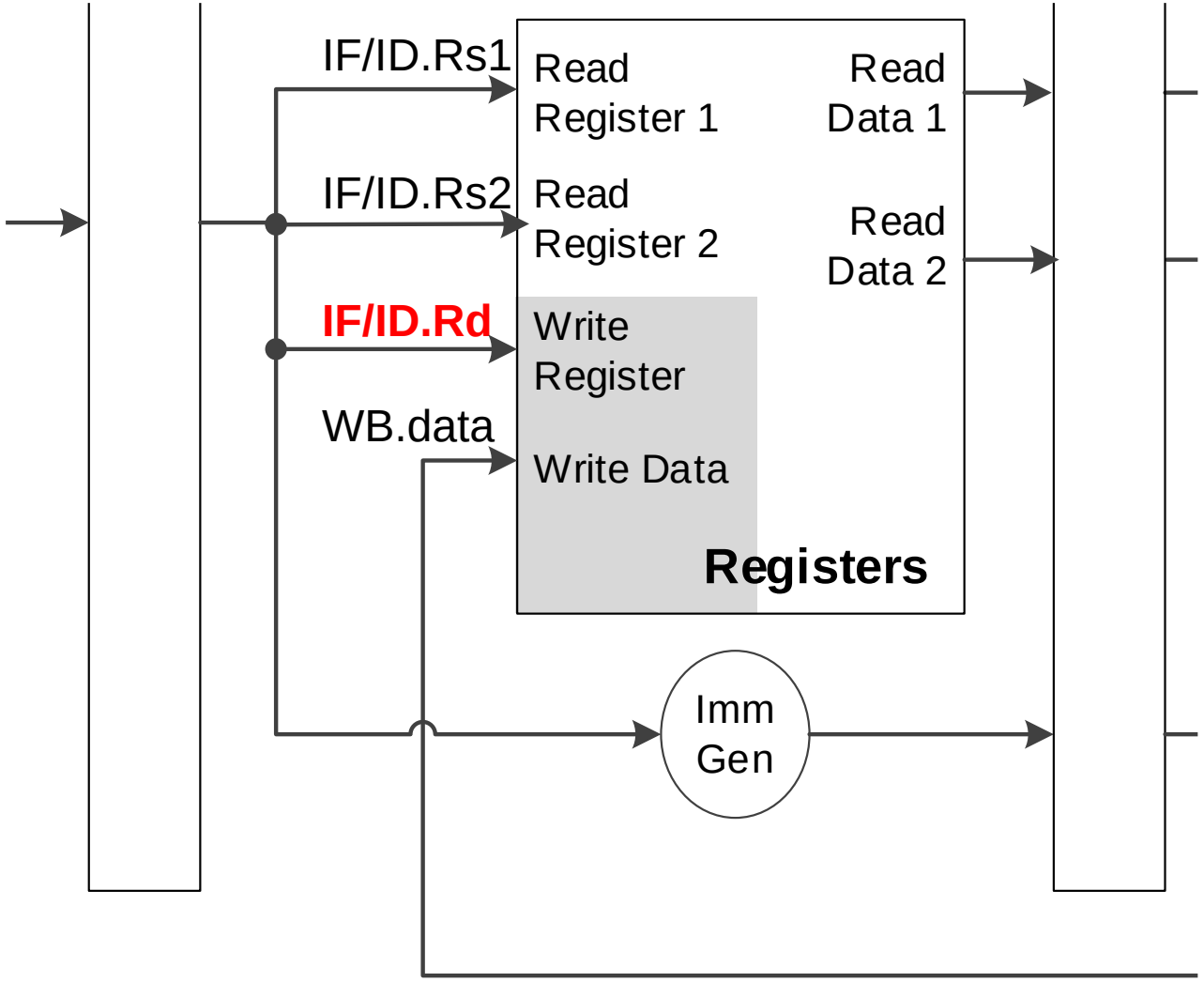
cycle 4



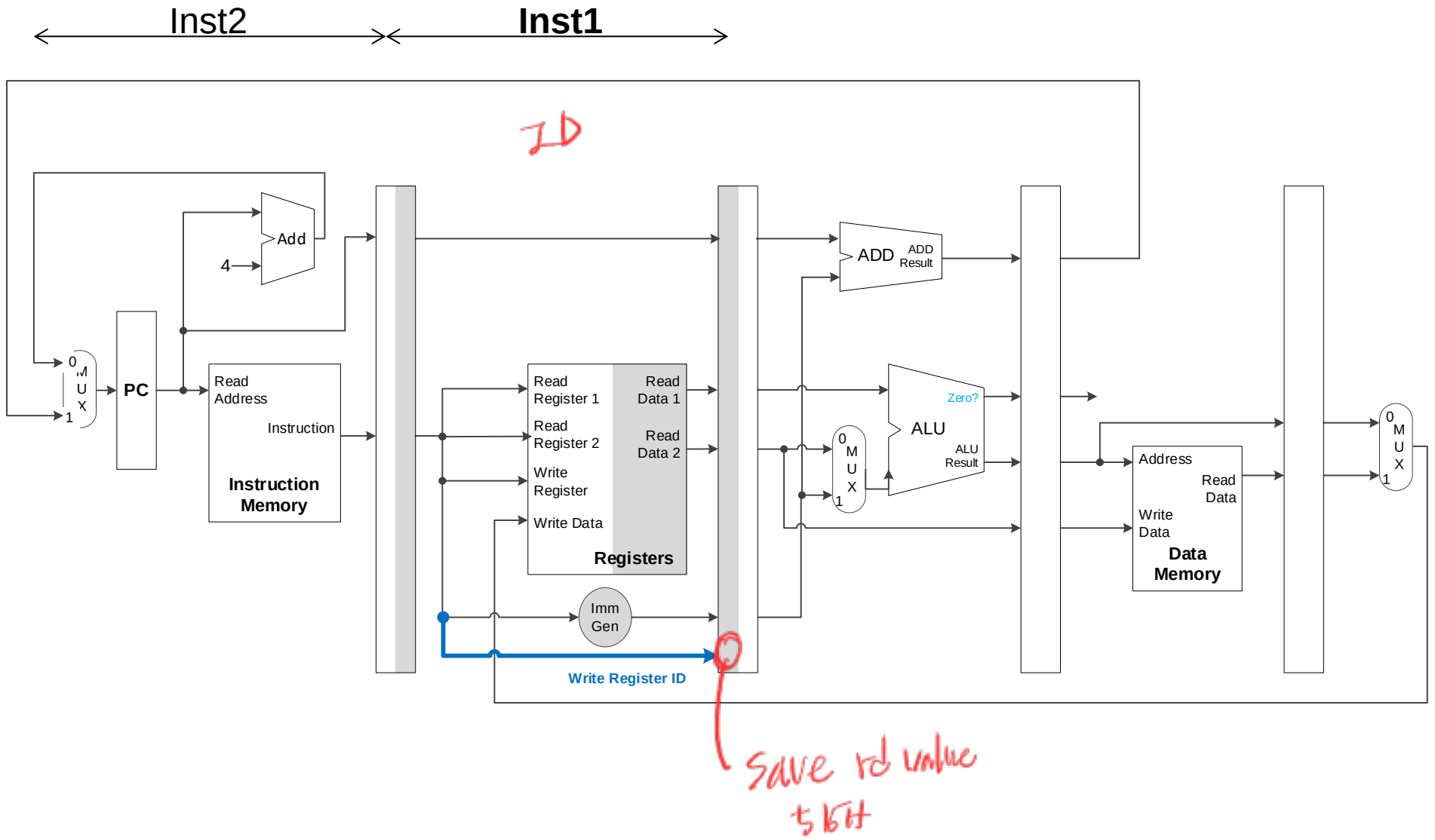
WB stage


Inst5
Inst4
Inst3
Inst2
Inst1
Add
4
M
U
X
PC
Read Address
Instruction
Instruction Memory
Read Register 1
Read Register 2
Write Register
Write Data
Read Data 1
Read Data 2
Registers
Imm Gen
ADD
ADD Result
Zero?
ALU
ALU Result
Address
Read Data
Write Data
Data Memory
?

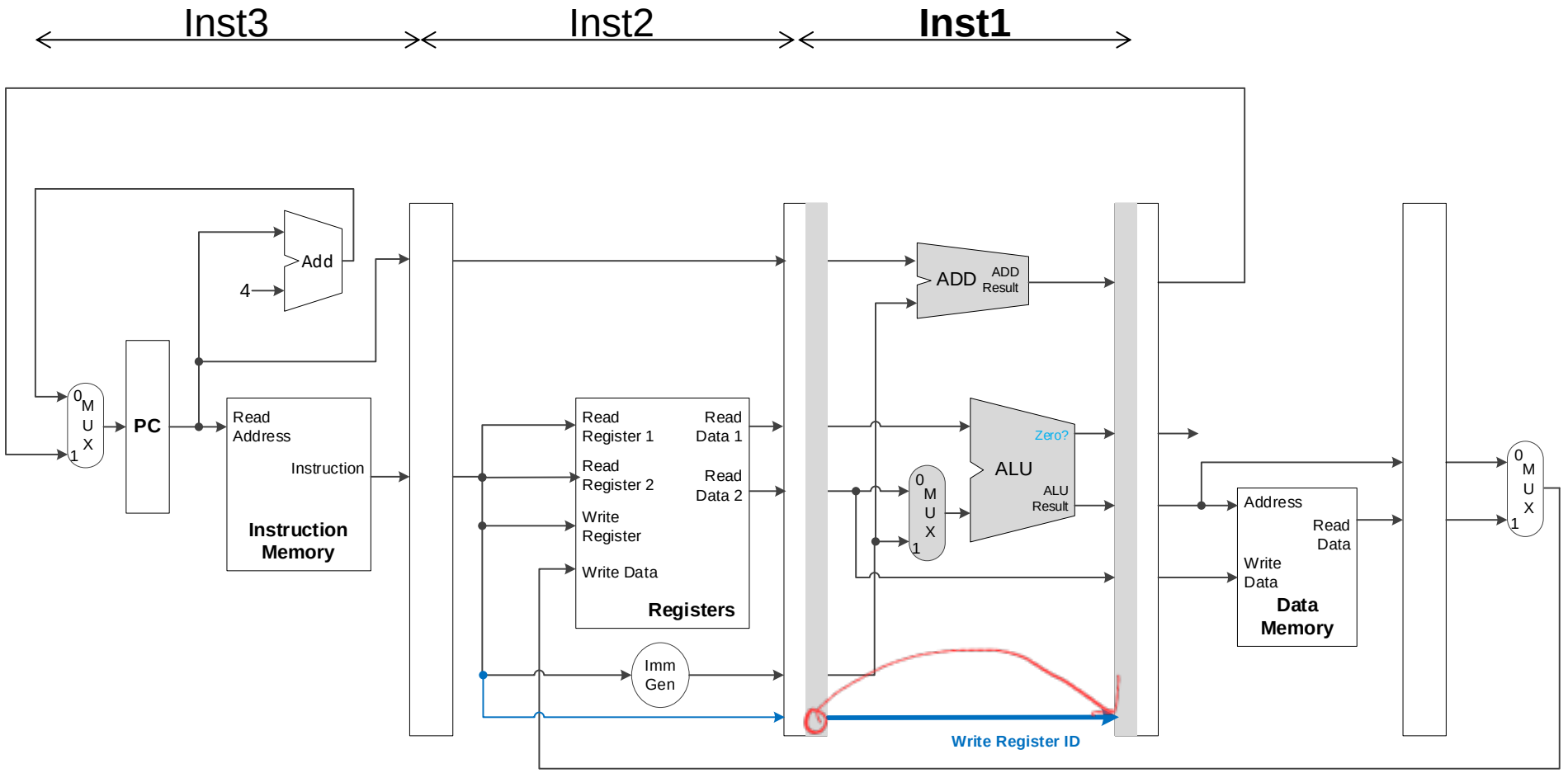
ID / WB stage


IF/ID.Rs1
Read
Register 1
Read
Data 1
IF/ID.Rs2
Read
Register 2
Read
Data 2
IF/ID.Rd
Write
Register
WB.data
Write Data
Registers
Imm
Gen

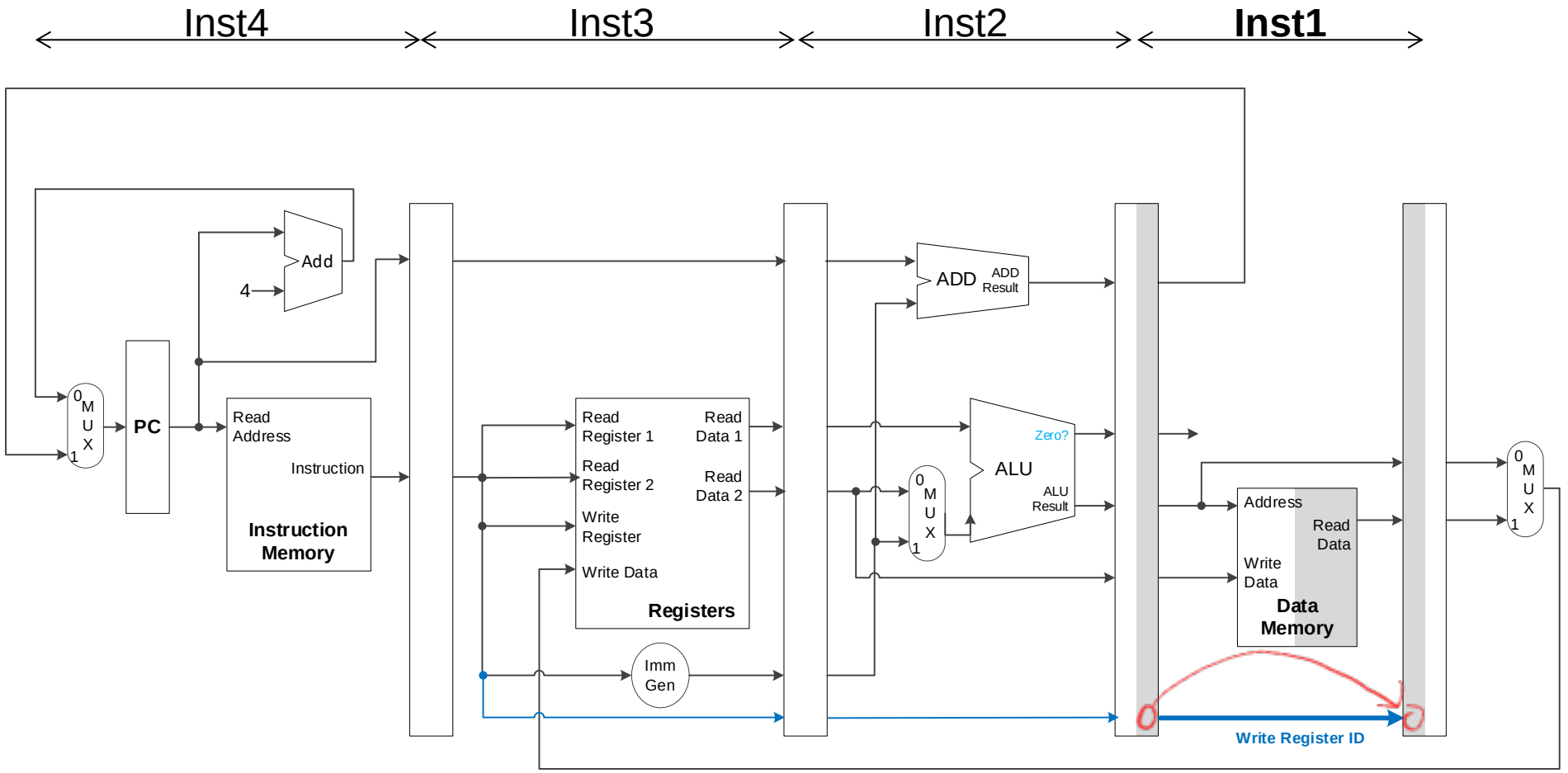
Corrected Datapath for Write Register ID



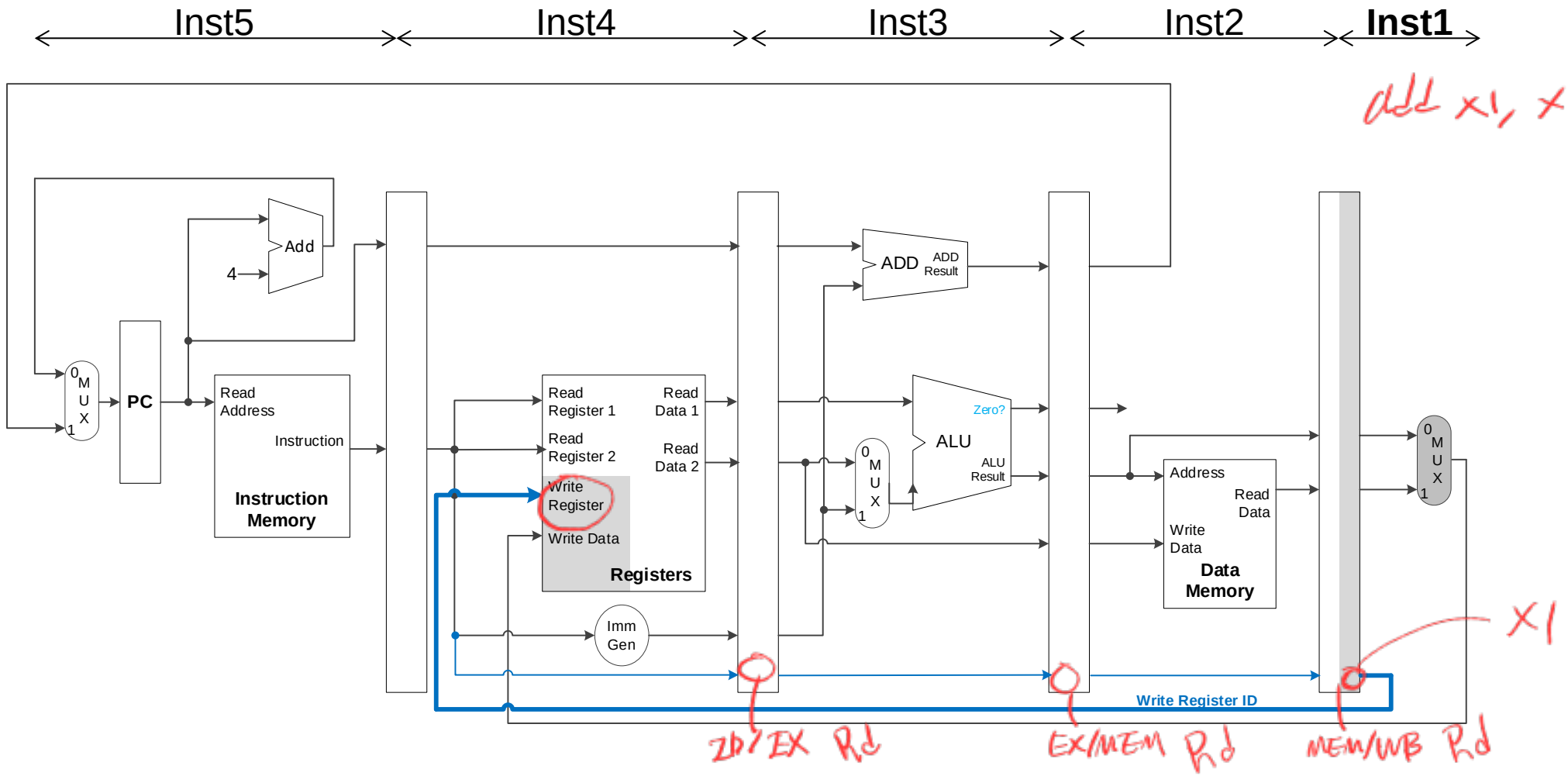
Corrected Datapath for Write Register ID



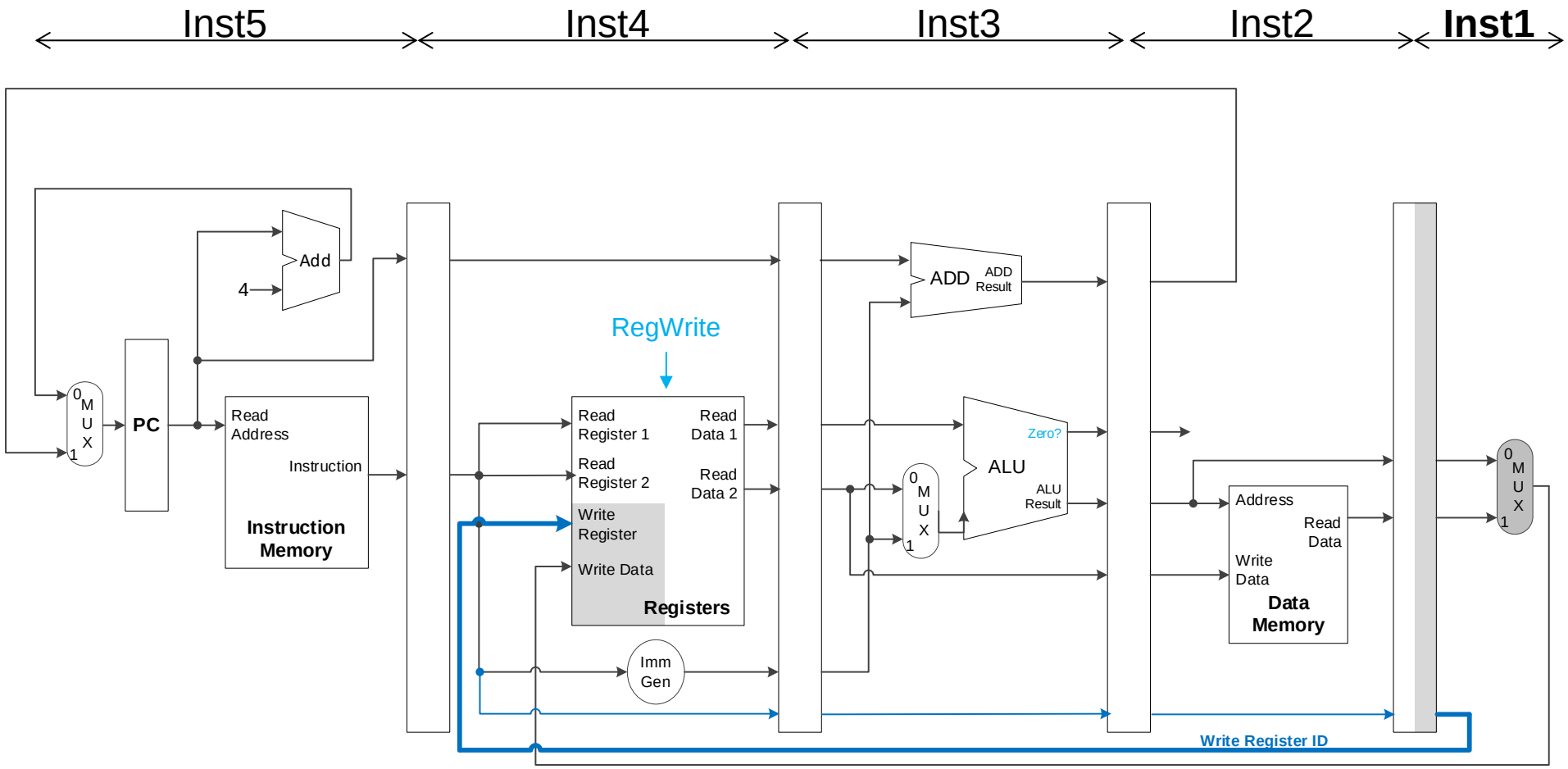
Corrected Datapath for Write Register ID



Corrected Datapath for Write Register ID

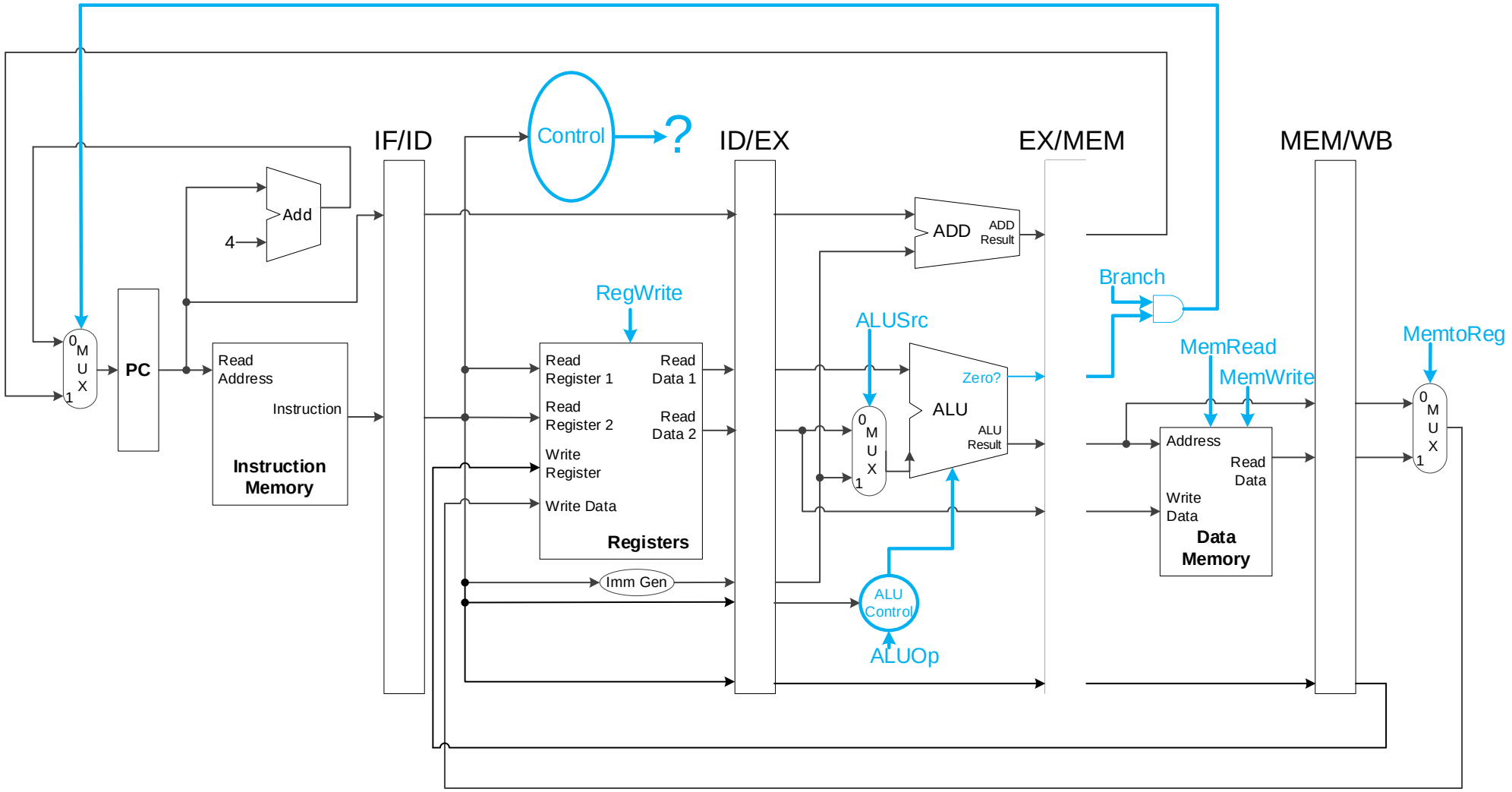


Control Signals?



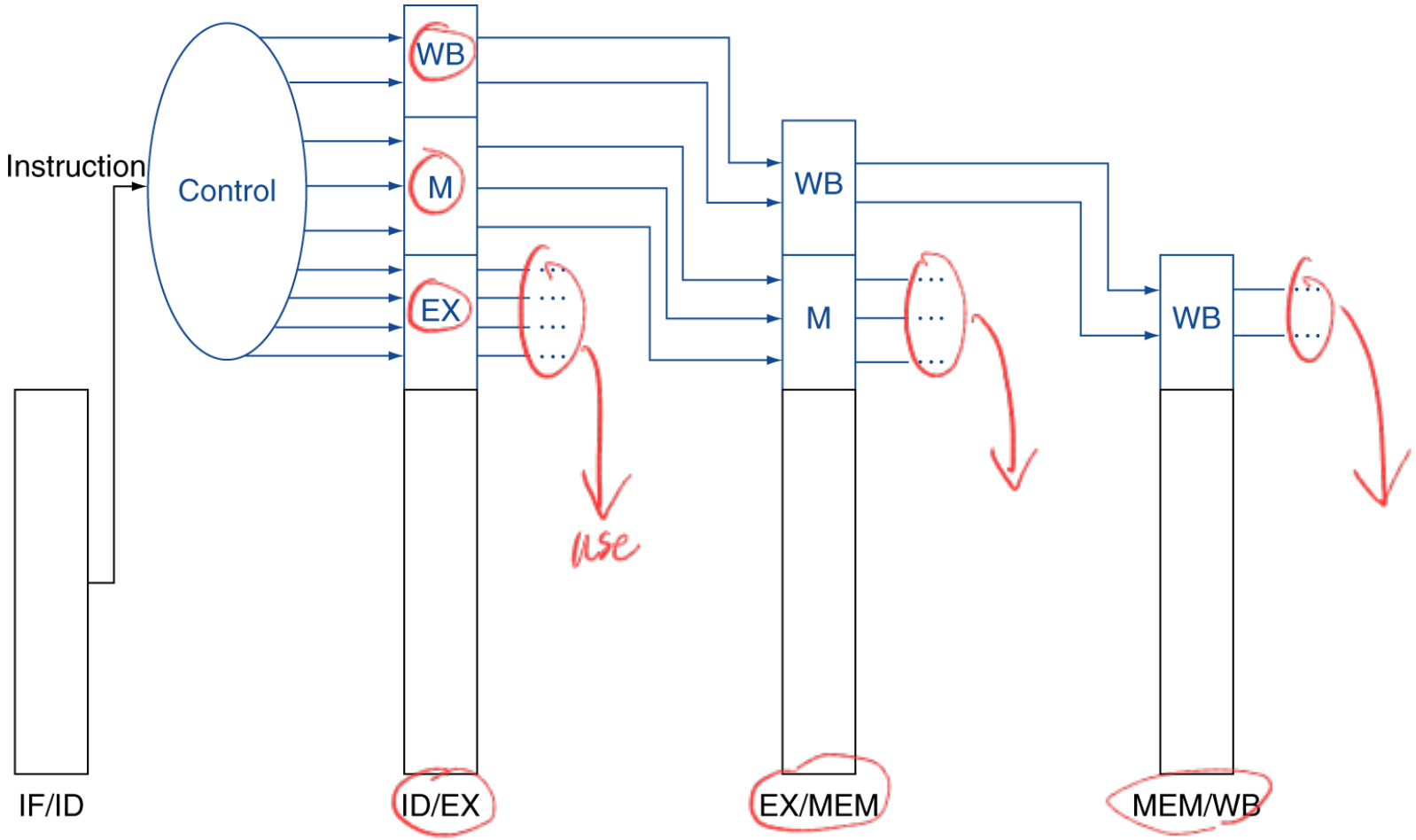
sw: Rfg RegWrite=0, add RegWrite=1 Controle 어떻게 지정?

Control Signals in Pipeline

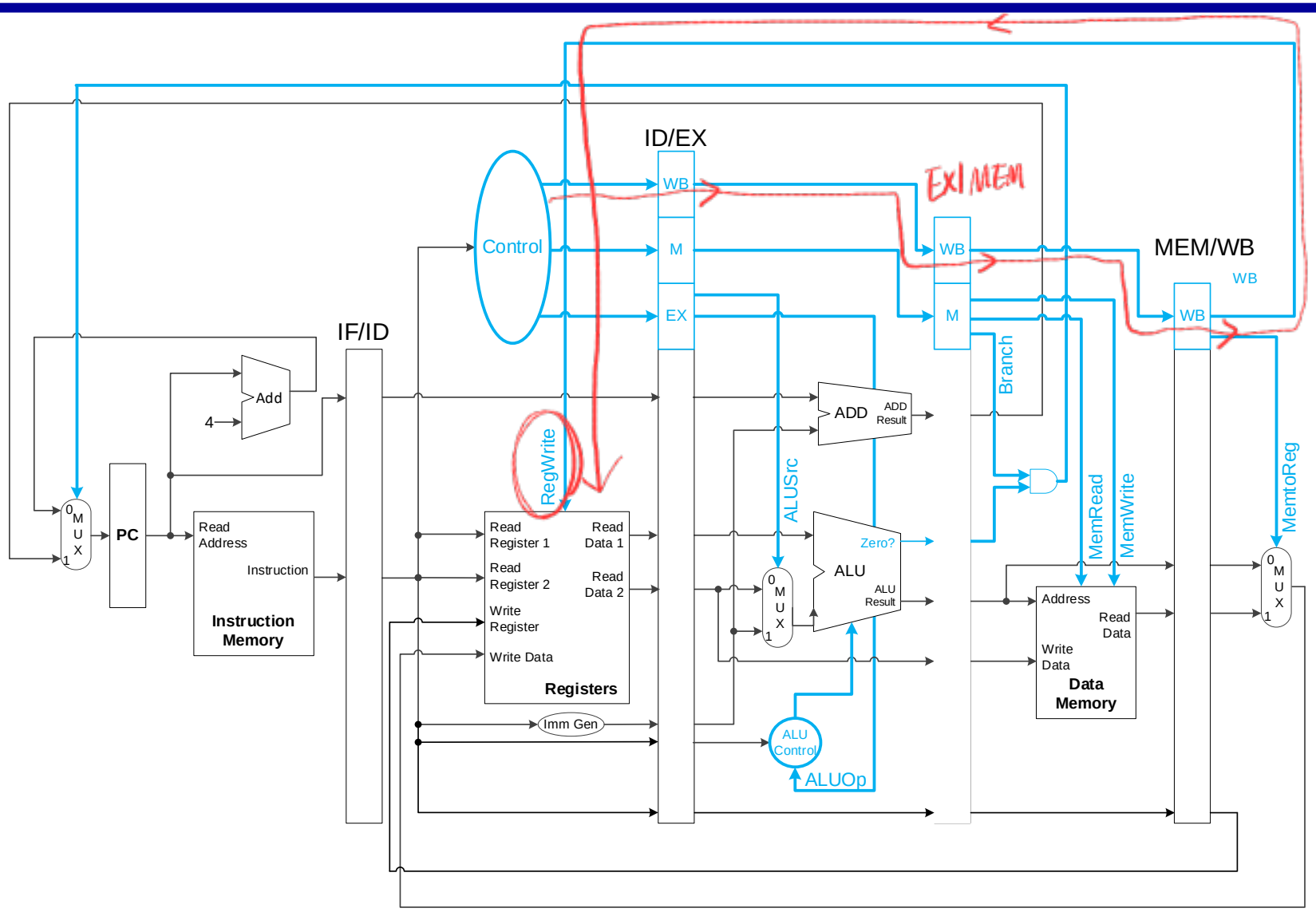


- Control signals derived from the opcode @ ID stage

Pipelined Control

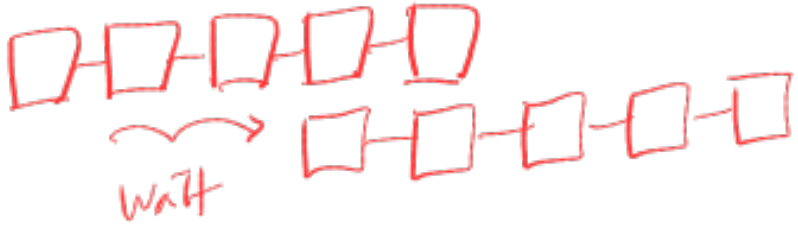


Pipelined Control



Hazards

- Hazard: Danger, risk, …



- Pipeline hazard:
  - A situation where the processor cannot process the next instruction in the following cycle.

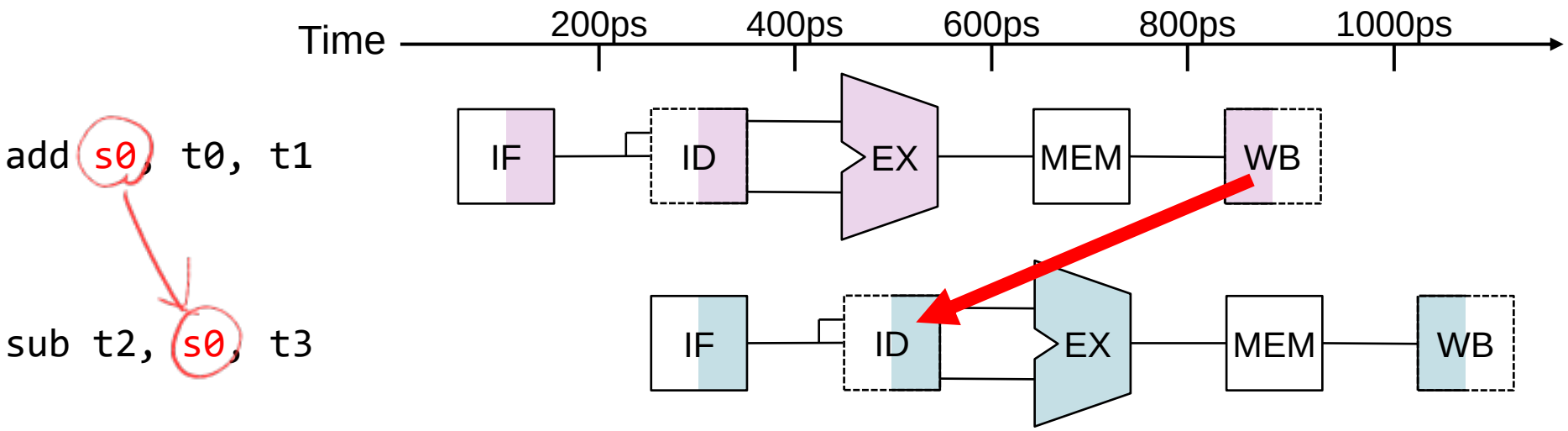
Types of Hazards

1. Structural hazards
   - A required resource is busy
   - Multiple instructions are trying to use the same component in the same cycle
2. Data hazards
   - Need to wait for previous instruction to complete its data read/write
   - Data value is not ready for the next instruction.
3. Control hazards
   - Deciding on control action depends on previous instruction
   - Similar to data hazard, but related to instructions that changes PC

Data Hazards

- Using a register value updated by the preceding instruction

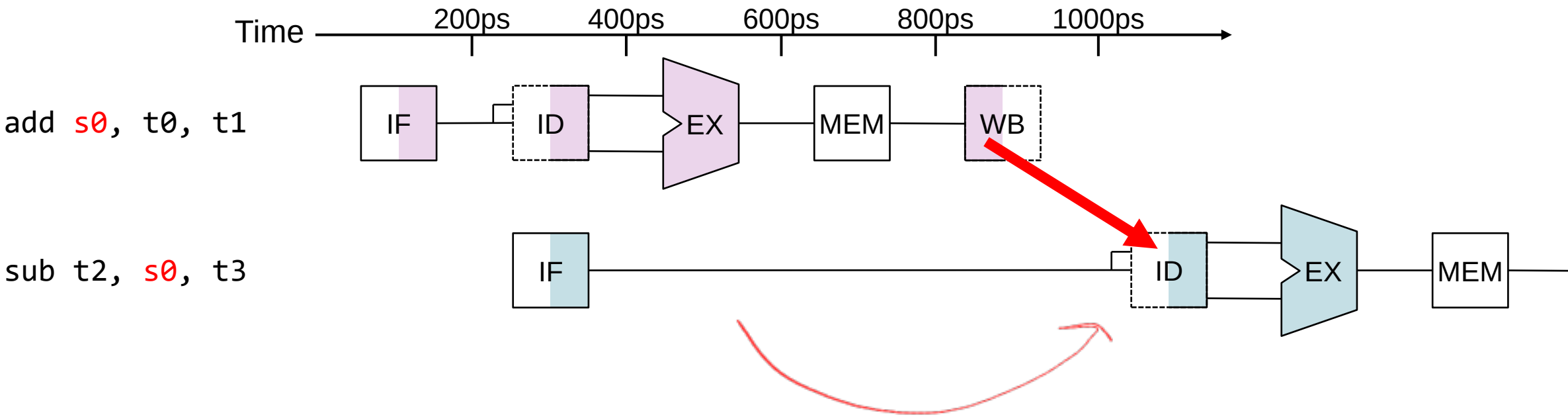
```
add     s0, t0, t1
sub     t2, s0, t3
```



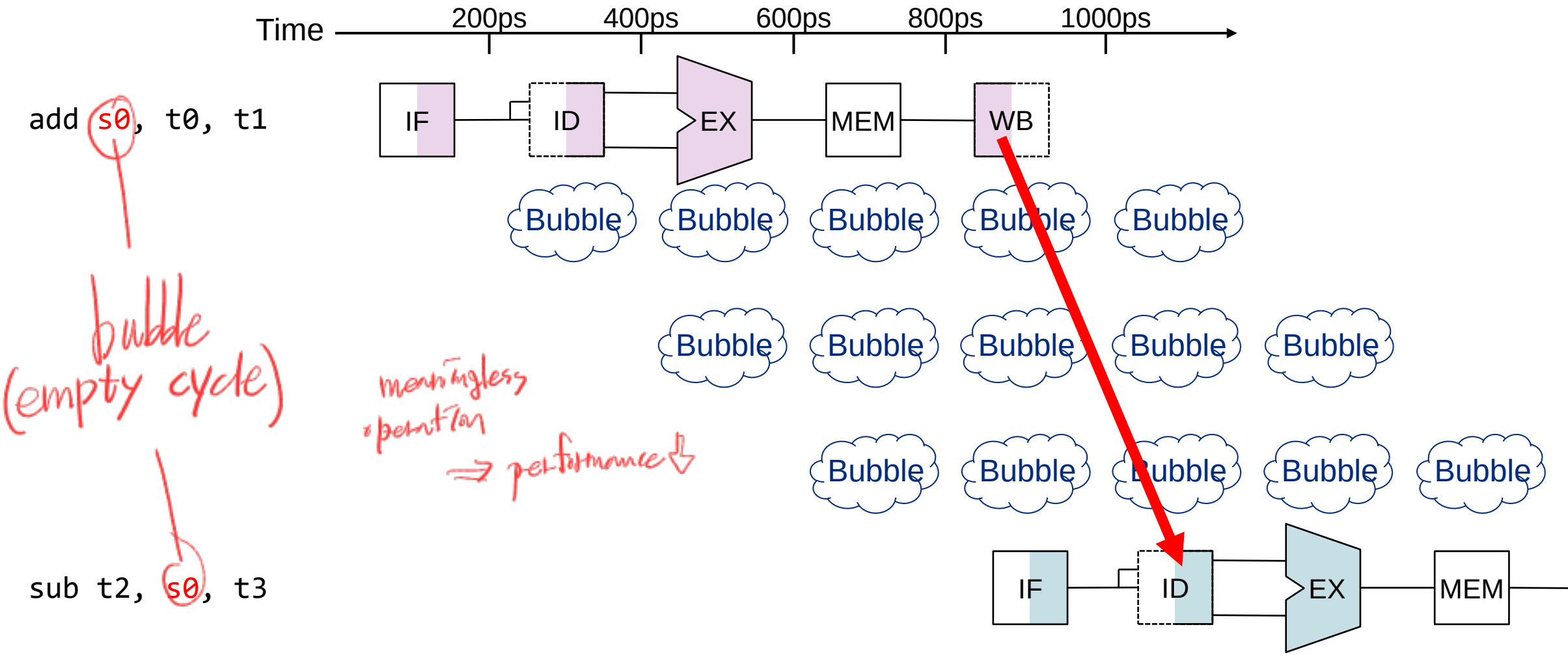
Data Hazards

- Using a register value updated by the preceding instruction

```
add     s0, t0, t1
sub     t2, s0, t3
```

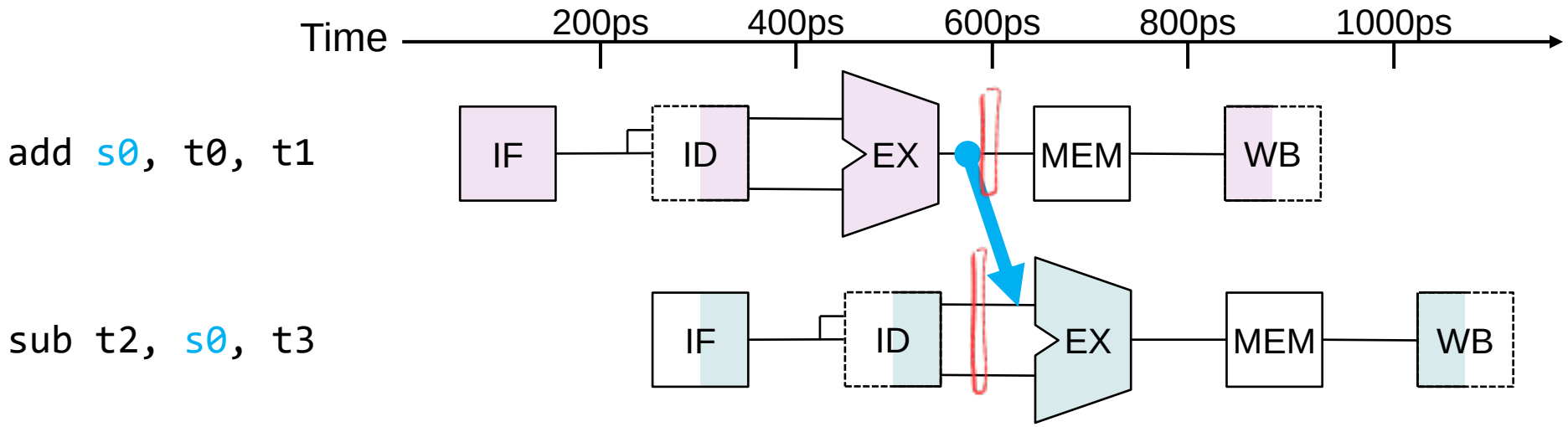


Pipeline Stall



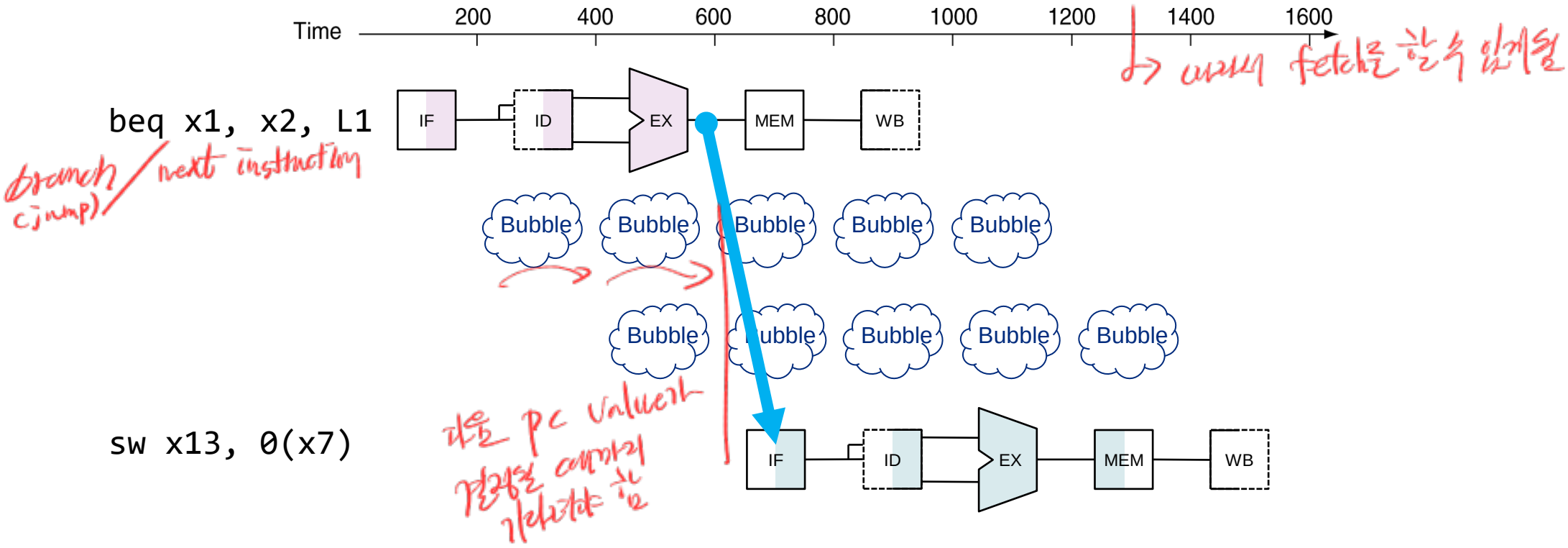
Forwarding

- Use result when it is computed
  - Don’t wait for it to be stored in a register
  - Requires extra connections in the datapath



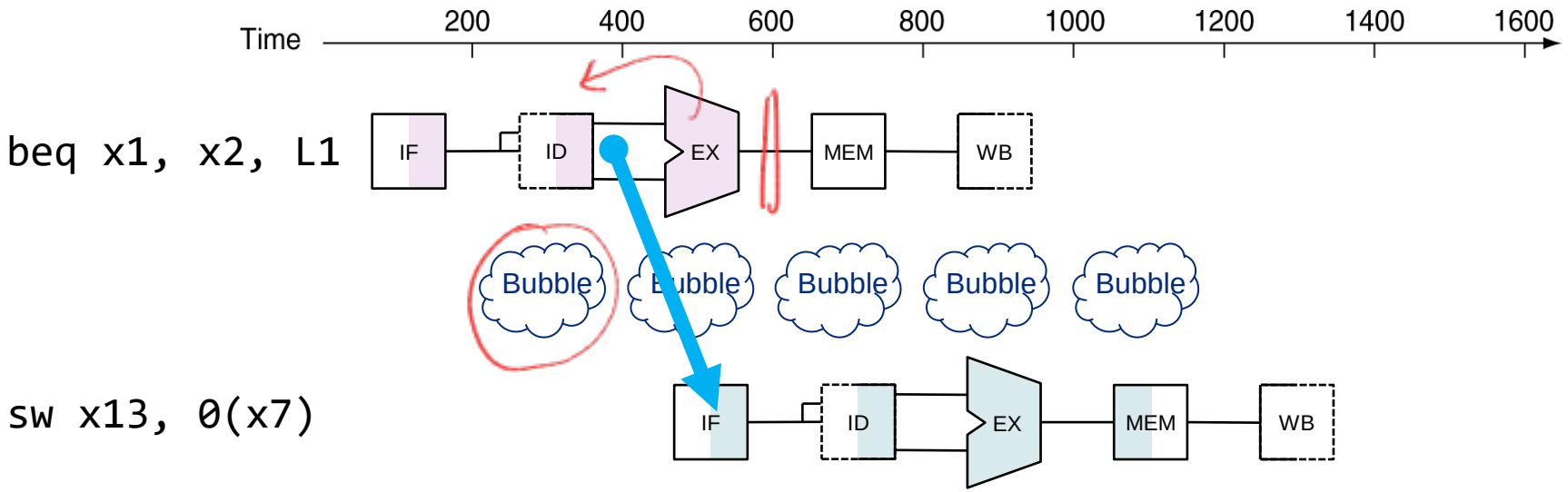
Control Hazards

- Branch determines flow of control
  - PC address should be detemined BEFORE the IF stage
  - Branch decision: EX stage (ALU)
  - Branch address: EX stage (Additional adder ALU)



Control Hazards

- The best we can do…
  - Add extra hardware resource in the ID stage
    - Compare the register values in the ID stage
    - Branch address calculation



  - Still need 1 cycle stall (Stall on branch)

Branch Prediction

- Longer pipelines can't determine branch outcome early
  - Intel i7: 14 pipeline stages
  - Stall penalty becomes unacceptable

pipelined 길어질수록 → penalty ↑
branch 결과를 빨리 알기 힘들어

- Predict outcome of branch
  - Stall only when prediction is wrong

accurate prediction ↑

Pipeline Summary

- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency

- Subject to hazards
  - Structure, data, control